

Bridging Evolutionary Algorithms and Reinforcement Learning: A Comprehensive Survey on Hybrid Algorithms

Pengyi Li , Jianye Hao , Senior Member, IEEE, Hongyao Tang , Xian Fu , Yan Zheng ,
Ke Tang , Fellow, IEEE

Abstract—Evolutionary Reinforcement Learning (ERL), which integrates Evolutionary Algorithms (EAs) and Reinforcement Learning (RL) for optimization, has demonstrated remarkable performance advancements. By fusing both approaches, ERL has emerged as a promising research direction. This survey offers a comprehensive overview of the diverse research branches in ERL. Specifically, we systematically summarize recent advancements in related algorithms and identify three primary research directions: EA-assisted Optimization of RL, RL-assisted Optimization of EA, and synergistic optimization of EA and RL. Following that, we conduct an in-depth analysis of each research direction, organizing multiple research branches. We elucidate the problems that each branch aims to tackle and how the integration of EAs and RL addresses these challenges. In conclusion, we discuss potential challenges and prospective future research directions across various research directions. To facilitate researchers in delving into ERL, we organize the algorithms and codes involved on <https://github.com/yeshenpy/Awesome-Evolutionary-Reinforcement-Learning>

Index Terms—Evolutionary Algorithms, Reinforcement Learning, Evolutionary Reinforcement Learning.

I. INTRODUCTION

REINFORCEMENT Learning (RL) [1] is an important category of learning methods within the field of machine learning [2], specializing in solving various sequential decision-making problems. By modeling sequential decision-making problems as Markov Decision Processes (MDPs) [3], RL learns an optimal policy via iterative policy optimization and evaluation with various optimization techniques (e.g., gradient descent [4]). Thanks to the development of deep learning, the capabilities of RL have been further enhanced. By leveraging the powerful expressive capabilities of neural networks and efficient gradient optimization, RL can approximate more complex value functions and demonstrate superior

learning efficiency compared to gradient-free methods [5]. In addition, RL, especially off-policy RL, collects and reuses historical samples, significantly improving sample efficiency and making it more applicable to problems where samples are costly [6]. With these advancements, RL has achieved significant successes in diverse domains, including Game AI [7], Robotics [8], Recommender System [9], and Scheduling Problems [10]. However, RL still faces several inherent and long-standing challenges, including limited exploration abilities [11], poor convergence [12], sensitivity to hyperparameters [13], and suboptimality in gradient optimization [14]. These challenges hinder the application of RL in more complex real-world scenarios.

Evolutionary Algorithms (EAs) [15]–[19] are a class of gradient-free, black-box optimization methods. By emulating Darwin’s theory of evolution, EAs iteratively evolve solutions. Due to the diversity within populations and the gradient-free random search, EAs have strong exploration ability. As a result, compared to conventional local search algorithms like gradient descent, EAs exhibit better global optimization capabilities within the solution space and are adept at solving multimodal problems [17], [20]. Moreover, EAs show good robustness and convergence properties, displaying resistance to noise and uncertainty [21]. With these characteristics, EAs have demonstrated formidable capabilities in various practical optimization problems [22], [23], including path planning [24], scheduling problems [25], and circuit design [26]. Besides, EAs have also demonstrated the capacity to evolve a single policy for solving multi-task sequential decision problems [27]–[29]. Nonetheless, EAs also have weaknesses, including sensitivity to the design and selection of variation operators [30], as well as ineffective and redundant exploration arising from random search [5]. Besides, EAs often demonstrate low sample efficiency in sequential decision-making problems [12], [31], [32].

As discussed above, RL and EAs are two categories of methods based on different principles. Each has proven its efficiency in addressing specific problems but also faces different challenges. With the development of the EAs and RL communities, many researchers have found that these challenges can be addressed by combining advanced methods from both fields, leading to the emergence of numerous hybrid methods. For brevity, we refer to these works as Evolutionary Reinforcement Learning (ERL). However, the technical pathways of these methods and the problems they aim to address

This work is supported by the National Natural Science Foundation of China (Grant Nos. 92370132, 62106172), the Xiaomi Young Talents Program of Xiaomi Foundation, the National Project X of China (Grant No. JCKY2021204B104), and the Science and Technology on Information Systems Engineering Laboratory (Grant Nos. WZC20235250409, 6142101220304). (Corresponding author: Jianye Hao.)

Pengyi Li, Jianye Hao, Xian Fu, and Yan Zheng are with the College of Intelligence and Computing, Tianjin University, Tianjin 300350, China (e-mail: lipengyi@tju.edu.cn; jianye.hao@tju.edu.cn; xianfu@tju.edu.cn; yanzheng@tju.edu.cn).

Hongyao Tang is with the Montreal Institute of Learning Algorithms (MILA), Quebec H3A 0G4, Canada (e-mail: tang.hongyao@mila.quebec).

Ke Tang is with the Department of Computer Science and Engineering, Southern University of Science and Technology, Shenzhen 518055, China (e-mail: tangk3@sustech.edu.cn)

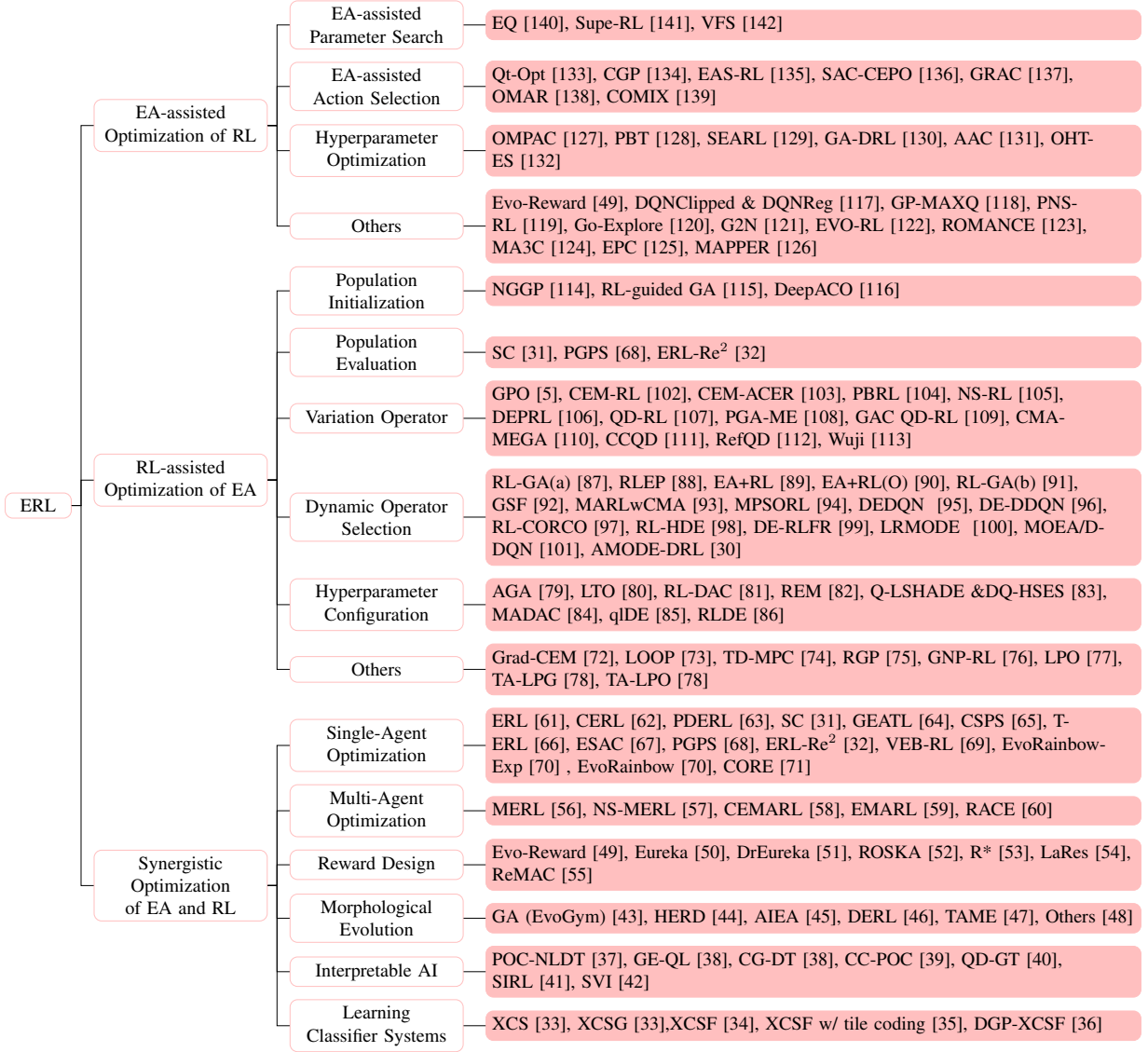


Fig. 1. Three major research directions in the ERL field. Each direction comprises multiple research branches.

are various and diverse. There is a lack of a comprehensive survey and systematic analysis in the ERL literature. In this paper, we attempt to provide a systematic review of existing ERL works. We first revisit these works from the following three primary perspectives:

① **From the perspective of RL:** RL is a class of learning algorithms used to tackle various sequential decision-making problems. Apart from modeling problems as MDPs, RL involves configuring algorithms, interaction, and learning and optimizing network parameters. This process encounters many optimization challenges, such as hyperparameter optimization, action selection, and network parameter optimization. EAs exhibit strong search capabilities and global optimization prowess, which can further enhance the quality of solving such optimization sub-problems within RL.

② **From the perspective of EAs:** EAs are a category of optimization algorithms that require iterative searches in the solution space to obtain feasible solutions. This typically involves population initialization, evaluation, operator design

and selection, and algorithm configuration. However, this process often confronts assorted hurdles, such as how to construct the initial population, which often determines the quality of the final solution; how to perform efficient mutation that avoids redundant and inefficient exploration; and how to dynamically select operators at different stages of optimization to improve performance. The learning ability of RL enables it to develop strong discriminative guidance and decision-making capabilities. Incorporating RL into the optimization process of EAs has been proven to further enhance the efficiency of EAs.

③ **From the perspective of collaboration:** EAs and RL can collaborate to solve a problem, typically through two approaches: 1) EAs and RL simultaneously address the same problem and collaborate through some mechanisms. 2) EAs and RL each solve a part of the problem and eventually integrate to form a complete solution. The former approach aims to complement the strengths and weaknesses of EAs and RL in problem-solving: EAs' exploration capability compensates for RL's exploration limitations, while RL's experience reuse

and fine-grained learning address EAs' sample inefficiency. The latter approach involves EAs and RL tackling tasks they excel in individually; for instance, EAs optimizing topology and RL learning control policies.

Based on the insights mentioned above, we categorize the ERL works into three main directions: 1) **EA-assisted Optimization of RL**. This integration approach incorporates EAs into the learning process of RL, mainly applied to address sequential decision-making problems. It leverages diverse exploration, global optimization capabilities, and strong convergence and robustness of EAs to address challenges in RL such as limited exploration capabilities, sub-optimality in gradient optimization, and sensitivity to hyperparameters. 2) **RL-assisted Optimization of EA**. Opposite to the previous category, this integration approach incorporates RL into the optimization flow of EAs, mainly applied to address various optimization problems and sequential decision-making problems. It leverages efficient experience utilization and learning capabilities, along with the discriminative abilities of RL to address challenges in EAs such as population initialization, algorithm configuration, uncontrollable mutation, and high sample cost in fitness evaluation. 3) **Synergistic Optimization of EA and RL**. This integrated approach maintains complete processes for both EAs and RL to collaboratively address the same problem simultaneously or independently tackle partial solutions, which are subsequently integrated into a complete solution. This allows each of them to leverage their respective strengths in problem-solving and mutually enhance each other through complementary features, ultimately achieving better performance.

There have been some efforts to review works related to EAs and RL, including comparing EAs with RL algorithms [143], exploring the integration of EAs and RL for policy search [12], [144], reviewing the hybrid algorithms within a unified framework, including motivation, natural models, sub-algorithms, techniques, and properties [145], reviewing hybrid algorithms based on the challenges they address in RL [146], and reviewing hybrid algorithms according to the key research areas of RL [147]. However, these surveys lack comprehensive and systematic investigation into hybrid algorithms. Thus we aim to furnish a more systematic and comprehensive survey to fill this gap. We summarize our contributions as follows:

- We conduct a thorough and systematic analysis of the works in the ERL domain, leading to the establishment of three research directions: EA-assisted Optimization of RL, RL-assisted Optimization of EA, and Synergistic Optimization of EA and RL.
- In each direction, we further subdivide the works into distinct research branches. Subsequently, we conduct an in-depth analysis of the fundamental issues to be addressed and the related algorithms for each branch.
- Furthermore, we point out open challenges within each domain and propose potential research directions to address these challenges.

We begin by providing an overview of the basic algorithms in EAs and RL and definitions of involved problems in Section II. Subsequently, we delve into each direction to

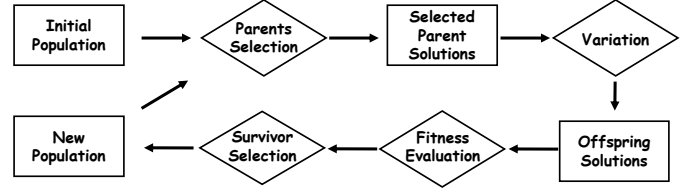


Fig. 2. Evolutionary Algorithm optimization process. Diamond-shaped blocks represent the actions taken by the algorithm, while rectangular blocks represent instances generated by the actions.

categorize the related works and present them from various branches, outlining the specific problems they address and the proposed approaches. After presenting the various branches within each research direction, we summarize open challenges and discuss potential future directions.

II. BACKGROUND

A. Evolutionary Algorithms

Evolutionary Algorithms (EAs) are a class of biologically inspired gradient-free optimization methods that emulate biological evolution processes [15], [17], [19]. Figure 2 illustrates the entire evolutionary process. EAs begin by initializing the population with an initial set of individuals $\mathbb{P} = \{I_1, I_2, \dots, I_N\}$. These individuals can have various forms in different problems, such as vectors, neural networks, and so on. Subsequently, EAs select parents using a selection operator, and offspring individuals are generated through variation. The selection operator typically involves choosing individuals with the highest fitness scores determined through evaluation. Following this, the offspring are evaluated for their fitness, followed by survivor selection. The selected individuals form the new population for the next generation. All the EAs can be abstracted into the aforementioned process. In this survey, we cover various EAs [19], such as Genetic Algorithm (GA) [23], [148], Differential Evolution (DE) [149], Evolution Strategy (ES) [150], Novelty Search (NS) [151], MAP-Elites [108], [152], Genetic Programming (GP) [153], among others. Fixed genomes are a common feature across GA, DE, and ES. In ES and DE, genomes are typically represented as real-valued vectors, whereas GA utilizes binary strings. Recently some works have expanded the forms of these methods' genomes, allowing them to employ neural networks as genomes [12]. While NS and MAP-Elites are two diversity mechanisms. GP stands out for its ability to handle variable-length genomes, such as tree structures, which enables the exploration of complex topology. It is worth noting that in sequential decision-making problems, existing EAs typically utilize the policies, i.e., neural networks, as individuals in the population, and rely on these policies for action decision-making and interaction. Ultimately, they evaluate the population based on the averaged cumulative rewards obtained throughout several episodes of games, followed by evolution in the parameter space [32], [102]. We can observe that EAs struggle to utilize finer-grained information, such as states, actions, and step-level rewards. This is one major factor leading to sample inefficiency.

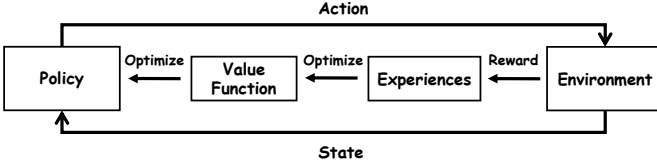


Fig. 3. Reinforcement Learning process.

B. Reinforcement Learning

Reinforcement Learning (RL) needs to formalize the target problem as Markov Decision Processes (MDPs) [1], where the agent interacts with the environment over several finite time steps. The MDP can be defined by a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma, T)$, where $\mathcal{S}$ denotes the state space, $\mathcal{A}$ is the action space, $\mathcal{P} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ is the state transition function, $\mathcal{R} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the reward function, γ is the discount factor, and T is the maximum episode length. At each time step t , the agent chooses an action $a_t \in \mathcal{A}$ according to the state $s_t \in \mathcal{S}$ and its policy $\pi(s) : \mathcal{S} \rightarrow \mathcal{A}$, receives a reward $r_t \in \mathcal{R}(s_t, a_t)$, and gets the next state s_{t+1} based on $\mathcal{P}(s_{t+1}|s_t, a_t)$. RL aims to find a policy that maximizes the cumulative discounted rewards $R_t = \sum_{i=t}^T \gamma^{i-t} r_i$ at each time step t . Q-learning [154] is a classical value-based RL algorithm that uses a Q-function $Q(s, a)$ to learn the Q-value, i.e., the cumulative reward obtained from taking a specific action a in a given state s . Q-learning selects actions based on the maximum Q-value $\pi = \arg \max_a Q(s, a)$ and employs the Bellman equation to update the Q-function towards the target y using $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha(y - Q(s_t, a_t))$ and $y = \sum_{i=t}^{i+n-1} \gamma^{i-t} r_i + \gamma^n \max_a Q(s_{t+n}, a)$, where n denotes the number of steps considered for future rewards, larger values of n improve the accuracy of the value function approximation by incorporating more reward signals. However, this also leads to increased variance. This approach is referred to as n -step TD, where n is often set to 1, considering only the one-step reward. Policy Gradient methods [155], [156] are a class of policy-based RL algorithms that directly optimize the policy of an agent. The basic idea is to update these parameters in the direction that increases the expected cumulative reward $\bar{R}_\theta = \mathbb{E}_\tau[R(\tau)]$, which is often done using techniques like gradient ascent. One classical algorithm is REINFORCE [155], which computes the gradient of $\bar{R}_\theta$ for the policy parameters using $\nabla \bar{R}_\theta = \frac{1}{M} \sum_{k=1}^M \sum_{t=1}^T G_t^k \nabla \log \pi_\theta(a_t^k | s_t^k)$ and $G_t^k = \sum_{i=t}^T \gamma^{i-t} r_i^k$, where G_t^k represents the cumulative discounted rewards from a specific time step t until the end of episode k . However, REINFORCE can only conduct policy improvement after completing one episode, which leads to inefficiency. To solve the problem, the Actor-Critic method (AC) [157] integrates value-based RL into policy gradient updates. This is accomplished by training a critic to supply one-step gradients for the policy, i.e., the actor. The development of deep learning has further enhanced the capabilities of RL, leading to the proposal of many Deep RL algorithms, including DQN [158], [159], DDPG [160], TD3 [161], SAC [162], TRPO [163], PPO [164], and others.

C. Problem Definition

Based on the tasks addressed by the hybrid algorithms, we categorize them into five major classes: sequential decision-making problems, continuous optimization problems, combinatorial optimization problems, multi-objective optimization problems, and multimodal optimization problems. **Sequential Decision-making Problems (SDP)** involves modeling the task as a MDP. Through policy control, the agent interacts with the environment and receives rewards. The ultimate goal is to obtain a policy that maximizes cumulative rewards. Note that other optimization problems can also be modeled as sequential decision problems. For better distinction, we specify the primary tasks involved here: Maze Navigation Problems and Robot Control Problems, including MuJoCo [165] and DMC [166]. Additionally, we also explore multi-agent settings, which require training multiple policies to control multiple agents interacting in the environment, aiming to maximize team rewards. The related tasks covered in this paper include SMAC [167], MAMuJoCo [168], MPE [169], and flocking tasks [59]. In addition, this survey involves the other four types of optimization problems, including **continuous optimization problems (CTOP)** [170], **combinatorial optimization problems (COP)** [171], **multi-objective optimization problems (MOOP)**, **multimodal optimization problems (MMOP)**. CTOP and COP require finding a set of variables x in continuous or discrete spaces to either maximize or minimize the objective function $f(x)$. Typically, such problems come with constraints. CTOP primarily involves the CEC benchmark [172]–[174], while COP involves the Traveling Salesman Problem (TSP) [175], Vehicle Routing Problem (VRP) [176], and Scheduling Problems (SP) [177], among others. MOOP involves multiple objective functions, rather than a single one. In MOOP, our goal shifts from seeking a solution to minimize or maximize a single objective function, to finding a set of solutions where each represents an optimal solution across different preferences. Unlike MOOP, MMOP deals with situations in single or multi-objective optimization where a single objective function has multiple local optima. The objective of MMOP is to find all or as many local sub-optimal solutions as possible.

III. EA-ASSISTED OPTIMIZATION OF RL

This section offers a comprehensive analysis of how EAs can enhance RL. In this optimization process, the related algorithms revolve around RL, with EAs playing a supporting role in refining this process. EAs cannot solve the problem independently. The optimization schematic is illustrated in Figure 4a. These works primarily focus on sequential decision-making problems, which are the focus of most RL algorithms.

According to the optimization objective of EA, we classify the related methods into four branches: EA-assisted Parameter Search, EA-assisted Action Selection, Hyperparameter Optimization, and Others. Specifically, EA-assisted Parameter Search focuses on leveraging the optimization characteristics of EAs to further improve RL. EA-assisted Action Selection primarily focuses on the challenge of dealing with a vast action space in continuous action settings, where it is difficult

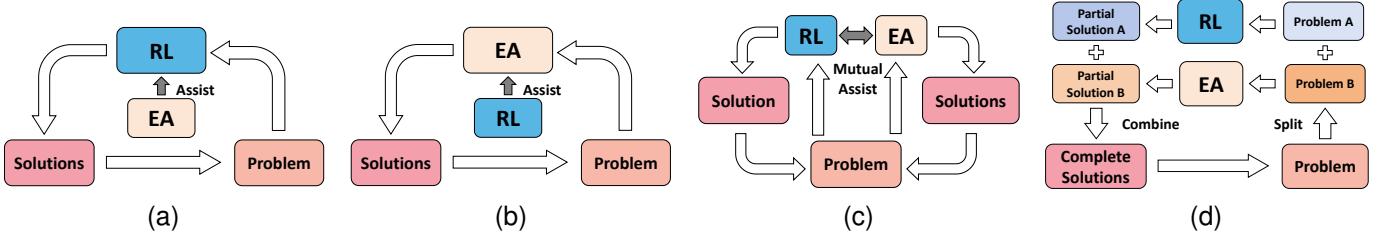


Fig. 4. Schematic of Four Integration Approaches: (a) EA-assisted Optimization of RL. RL conducts search and improvement of the solution, with EAs playing a supporting role; EAs cannot independently optimize solutions. (b) RL-assisted Optimization of EA. EAs conduct search and improvement of the solution, with RL playing a supporting role; RL cannot independently optimize solutions. (c) and (d) Synergistic Optimization of EA and RL.

to determine the optimal actions. The goal is to utilize EAs for action search to improve the optimization and decision-making process of RL. Hyperparameter Optimization focuses on using EAs to automatically adjust RL hyperparameters to mitigate RL hyperparameter sensitivity and improve RL convergence performance. Others encompass the works that leverage EAs to enhance RL from different perspectives, e.g., reward function search, loss function search, and exploration. Below, we provide a detailed introduction to each branch and the related works involved.

EA-assisted Parameter Search. The ultimate objective of DRL is to train a policy network capable of selecting actions that maximize the cumulative rewards. However, RL with a single policy often exhibits weak exploration capabilities, and gradient-based optimization can easily lead neural networks into local optima, impeding the ability to achieve better performance [178]. To solve the problems, researchers propose integrating EAs with RL to assist parameter optimization.

EQ [140] employs EAs to replace the conventional Bellman equation for critic optimization. Specifically, EQ maintains a critic population, where each critic optimizes a corresponding actor with policy gradients. The fitness of each critic is based on the scores derived from the interactions between its respective actor and the environment. The critics with high fitness are selected as parents and perturbed with Gaussian noise to generate offspring. The experiments on the inverted pendulum task demonstrate that using EAs can achieve better guidance capabilities than using conventional Bellman optimization while maintaining comparable performance. Supe-RL [141] employs GA to auxiliary policy parameter search. At regular intervals, a population of policies is initialized by introducing perturbations to the current RL policy. The population interacts with the environment, and elite individuals soft update their parameters to the RL policy. In off-policy RL, Supe-RL incorporates the elite experiences collected in the genetic evaluation phase into the RL replay buffer. The experiments show that Supe-RL can enhance Rainbow and PPO on navigation tasks and the MuJoCo environment respectively. VFS [142] employs the same idea as Supe-RL but differs by the periodic construction of a critic population. The population is evolved by introducing perturbations of differing scales to the critics. Eventually, the critic with the least deviation from the true value function is chosen to replace the RL critic. The true value function is approximated using unbiased Monte Carlo estimation. VFS demonstrates

improvements across various algorithms on MuJoCo tasks, including PPO, DDPG, TD3, and SAC.

EA-assisted Action Selection. Action selection runs throughout the entire process of RL improvement and evaluation. It primarily involves optimizing the value function, i.e., computing target values, and interacting with the environment. However, on continuous action tasks, the action space can be vast, making it challenging to determine the optimal actions for optimization and execution, especially in situations involving multimodality or multiple peaks. Traditional RL often employs greedy policies or random sampling from distributions for action selection. However, this approach struggles to accurately capture the best behaviors. Therefore, some works propose the concept of action evolution: initializing a population of actions, evaluating their quality using Q values as fitness, and then selecting the elite action for optimization and interaction.

Qt-Opt [133] does not explicitly maintain a policy network, it initializes a population of random actions from the action space. Under the current state, Qt-Opt applies two iterations of the Cross-Entropy Method (CEM) [179] to the population, guided by the Value Function. The best action is then selected for the critic optimization and interaction. Qt-Opt demonstrates its efficiency in real-world robotic visual grasping tasks. While retaining the advantages of Qt-Opt, CGP [134] reduces computational burdens by introducing a policy to mimic actions sampled by CEM. In CGP, the critic’s training uses CEM-derived actions. Concurrently, a policy is trained using behavior cloning or policy gradients. CGP validates its efficiency on MuJoCo tasks. EAS-RL [135] uses a similar idea to CGP, yet it distinguishes itself in two aspects: (1) employing TD3 with the original optimization process and replacing CEM with PSO, and (2) integrating both behavior cloning and policy gradients to optimize the policy. EAS-RL outperforms many ERL-related works on MuJoCo tasks [61]–[63], [102]. SAC-CEPO [136] employs a stochastic policy, SAC, instead of a deterministic policy in CGP. SAC-CEPO divides SAC into two parts: a mean network and a deviation network. CEM is employed to select the best mean actions, while the mean network is trained through behavior cloning, and the deviation network is learned using the SAC policy gradient. Besides, the generation of the CEM population is no longer based on random sampling, but on sampling from a normal distribution based on learned mean and deviation networks. SAC-CEPO demonstrates improvements over SAC in MuJoCo tasks. GRAC [137] introduces three mechanisms

TABLE I
EA-ASSISTED OPTIMIZATION OF RL.

Class	Algorithm	Task	EA	RL	Detail
Assisted Parameter Search	EQ [140]	MuJoCo	GA	DDPG	Critic Parameter
	Supe-RL [141]	Navigation & MuJoCo	GA	PPO & Rainbow	Policy Parameter
	VFS [142]	MuJoCo	GA	DDPG & PPO & TD3	Critic Parameter
Assisted Action Selection	Qt-Opt [133]	Real Robot Control	CEM	Double Q	Critic Update; Interaction
	CGP [134]	MuJoCo	CEM	Double Q	Critic Update; Policy Update
	EAS-RL [135]	MuJoCo	PSO	TD3	Policy Update
	SAC-CEPO [136]	MuJoCo	CEM	SAC	Policy Update
	GRAC [137]	MuJoCo & Real Robot Control	CEM	Double Q-learning	Critic Update; Policy Update
	OMAR [138]	MPE & D4RL	CEM-like	MA-CQL & CQL	Critic Update; Interaction
	COMIX [139]	MPE & MA-MuJoCo	CEM	QMIX	Policy Update
Hyperparameter Optimization	OMPAC [127]	Atari & Tetris	GA	TD(λ) Sarsa(λ)	$\lambda, lr, \tau_a, \gamma$
	PBT [128]	DM Lab & Atari & StarCraft II	GA-like	UNREAL & FuN & A3C	lr , Entropy Cost, Unroll Length, Intrinsic Reward Cost
	SEARL [129]	MuJoCo	GA	TD3	Parameters, γ , Architecture, Activation
	GA-DRL [130]	Gym Robotics & AuroReach	GA	DDPG	$\gamma, lr, \epsilon, \tau, \eta$ (noise std)
	AAC [131]	DMC & Industrial Benchmark	GA	SAC	γ, h, k, a, c
	OHT-ES [132]	DMC	ES	TD3	n, lr
Others	DQNClipped & DQNReg [117]	MiniGrid & Atari	GA-like	DQN	Loss Function Search
	GP-MAXQ [118]	Foraging Task	GP	MAXQ	Hierarchy Search
	PNS-RL [119]	Gym Robotics & MuJoCo	NS	TD3	Improve Exploration
	Go-Explore [120]	Atari	GA-like	Imitation Learning	Improve Exploration
	G2N [121]	Atari & MuJoCo	GA	A2C or PPO	Activation of neurons
	EVO-RL [122]	Gym Control	GP	Q-learning & DQN & PPO	Special Setting
	ROMANCE [123]	SMAC	QD	QMIX	Robust MARL
	MA3C [124]	SMAC, Hallway & Traffic Junction & Gold Panner	GA-like	CMARL	Robust Communication
	EPC [125]	Particle-world Environment	GA-like	MADDPG	Large-Scale MARL
	MAPPER [126]	Grid world	GA-like	A2C	Improve Stability
	LPO [77]	Brax & MinAtar	ES	PPO	Accelerate and Improve Meta RL
	TA [78]	Grid world & Invaders & Brax & MinAtar	ES	LPO & LPG	Aware Learning Time Remaining

to improve Double Q-learning. We primarily focus on two mechanisms closely aligned with EAs: CEM Policy Improvement and Max-min Double Q-learning. Similar to previous methods, CEM Policy Improvement employs CEM to search for the optimal actions, the difference is that CEM Policy Improvement uses the Q-value differences between the optimal actions and the actions taken by the RL policy as advantages to increase the probability of the RL policy selecting the optimal actions. Max-min Double Q-learning is proposed to address the underestimation problem of double Q-learning. Specifically, GRAC obtains the CEM action and RL action based on the next state. Subsequently, it utilizes double Q networks to estimate the minimum Q value for the CEM action and the minimum Q value for the RL action. The higher of the two values is selected as the target value. GRAC demonstrates

significant performance gains in MuJoCo tasks and is also validated on real robots.

In the offline setting, the same problem is demonstrated to exist: policy gradient improvements tend to get stuck in local optima due to the complex nature of the value function landscape. To solve the problem, OMAR [138] employs a modified version of CEM to select the most optimal actions and uses behavior cloning to fine-tune the policy. The efficiency of OMAR is validated through experiments on the MPE and D4RL benchmarks. In the multi-agent setting, COMIX [139] aims to address the problem of QMIX’s inapplicability in continuous action spaces [167]. Within COMIX, QMIX is utilized for value function approximation, followed by CEM for action selection. The experiments demonstrate that COMIX outperforms MADDPG in MPE and MA-MuJoCo tasks.

EA-assisted Hyperparameter Optimization. While DRL has shown remarkable prowess across diverse domains, it still suffers from the notorious issue of hyperparameter sensitivity, resulting in a complex and expensive tuning process. Many works solve the problems by incorporating EAs to tune hyperparameters for RL.

OMPAC [127] employs GA for RL hyperparameter optimization, including the trace-decay rate λ , discount factor γ , learning rate lr , and temperature τ_a of softmax action selection. OMPAC trains a population of RL individuals with different hyperparameter configurations and evaluates individuals based on cumulative rewards. After a certain number of iterations, the hyperparameters of non-elite individuals are perturbed by adding Gaussian noise to generate offspring. OMPAC demonstrates significant performance improvements over the basic RL algorithms in Atari and Tetris tasks. PBT [128] introduces a more generalized optimization framework that can be applied to any neural network training process. Here, we focus solely on the RL aspect. Similar to OMPAC, PBT trains a population of policies with different hyperparameters. After a certain training period, the individuals with high fitness directly replace inferior ones by perturbing their hyperparameters or by resampling hyperparameters from predefined distributions. The efficiency of PBT is demonstrated across diverse domains encompassing DM Lab, Atari, and StarCraft II. SEARL [129] utilizes GA to dynamically adjust the parameters of RL, network architectures (layers, nodes, activation functions), and the learning rates of both actor and critic. Similar to PBT, SEARL trains a population of RL individuals and stores experiences generated during the population evaluation phase in a shared replay buffer. Subsequently, SEARL employs GA to add Gaussian noise to network parameters and modify network architecture, and hyperparameters to form a new population. Then SEARL optimizes the population through DRL based on the shared replay buffer. In four out of five MuJoCo tasks, SEARL outperforms PBT in terms of performance. GA-DRL [130] focuses on adjusting a wider range of hyperparameters, including the discount factor γ , learning rates lr for both the actor and critic, the soft update coefficient τ , the probability of selecting random actions ϵ , and the variance of the noise η . These hyperparameters are encoded using an 11-bit binary representation. GA-DRL ranks individuals based on the minimum number of episodes required for the robotic arm to achieve an 85% success rate. The experiments demonstrate that the algorithms optimized with hyperparameters discovered by GA-DRL can achieve better performance in most of the robotic arm control tasks. AAC [131] dynamically adjusts five hyperparameters: the discount factor γ , the coefficient for SAC entropy h , action duration k , and the number of single-step updates a and c for both the actor and critic. AAC maintains a population of actor-critic individuals with different hyperparameter configurations and a shared replay buffer. Fitness is defined as the mean of cumulative rewards over multiple episodes. Based on the fitness, the top 20% best and worst individuals are selected. Superior individuals replace inferior ones with hyperparameter perturbations. Ultimately, AAC demonstrates its efficiency on the DMC benchmark and Industrial benchmark [180]. OHT-ES [132] employs ES

to dynamically adjust RL hyperparameters, including the n parameter in the n -step TD and the learning rate (lr). OHT-ES samples hyperparameters from distributions maintained by ES and employs these hyperparameters to train several off-policy policies. These policies control agents interacting with the environment for evaluation, and the generated experiences are stored in a shared replay buffer for RL learning. Subsequently, the distributions of hyperparameters are updated. OHT-ES demonstrates significant improvements over the basic algorithm TD3 on the DMC benchmark.

Others. Some works use EAs or the principles of EAs to assist in other aspects of RL, which cannot be categorized into the three categories mentioned above.

Evo-Meta RL [117] uses EAs to search for the RL loss function capable of generalizing across diverse environments. Specifically, it transforms the computation process of the loss function into a computational graph and introduces mutations to the operation nodes within it. Evo-Meta RL operates in both inner and outer loops; the outer loop identifies the parent computational structures based on cumulative rewards to generate offspring, while the inner loop conducts gradient optimization based on the structures identified in the outer loop. Evo-Meta RL constructs DQNClipped and DQNReg based on the two discovered loss functions and demonstrates their superiority over DQN and DDQN on Atari and MiniGrid tasks.

GP-MAXQ [118] employs GP to explore the hierarchical structure of the hierarchical RL method MAXQ [181]. In GP-MAXQ, MAXQ learns policies based on the hierarchies derived from GP, GP explores appropriate hierarchies using MAXQ's outputs. Experimental results on the forgiveness tasks indicate that GP can search for more efficient hierarchical structures. PNS-RL [119] aims to improve the exploration capacity of RL. PNS-RL consists of multiple populations, each comprising multiple exploration policies and one guiding policy. Each exploration policy maintains an actor, critic, and replay buffer, and is optimized through policy gradients along with soft updates towards the guiding policy. Additionally, PNS-RL maintains an archive for selecting the guiding policy. The exploration policy that outperforms the average performance of policies in the archive is added, whereas policies in the archive performing notably worse than the added policy are removed. The most novel individual in the archive is selected as the guiding policy and shared across different populations. Novelty is measured based on the distance in the pre-defined behavioral descriptor space between the agent and its nearest k neighbors. The experiments on MuJoCo tasks show that PNS-RL outperforms PBT-TD3, P3S-TD3, CEM-TD3, and others. Go-Explore [120] also focuses on exploration issues. Although it does not utilize RL, it addresses the challenging exploration problem in RL and provides significant inspiration. Go-Explore builds an archive of trajectories, recording trajectories reaching different states. Then it selects the state from the archive that most likely leads to a new state, replicates the trajectory in the environment based on the archive, reaches that state, and starts random exploration from the state. If the newly reached state is not in the archive or reaches an existing state with a more optimal trajectory, the archive is

updated. Then the policy is learned directly through imitation learning. Go-Explore outperforms other RL algorithms and surpasses human performance in challenging exploration problems Montezuma and Pitfall. G2N [121] employs a binary GA population to control the activation of hidden neurons in the RL policy network, aiming to enhance the exploration capability. The experiments demonstrate that G2N can improve PPO and A2C on MuJoCo and Atari tasks. EvoRL [122] simulates an evolutionary process, considering that some behaviors are innate and can only be obtained through evolution, while others are learnable. Therefore, EvoRL defines part of the policy’s behavior as learnable and employs RL for learning, while another part is represented using behavior trees and can only be evolved through GP.

In addition to the aforementioned works in the single-agent settings, some works focus on improving MARL with EAs. ROMANCE [123] focuses on the robust MARL. ROMANCE utilizes the QD algorithm to maintain a population of attackers that sporadically attack some collaborators within the team. The attackers employ policy gradients for individual mutation, incorporating regularization terms for maintaining population diversity and attack frequency. ROMANCE demonstrates its superiority over other robust MARL algorithms on the SMAC benchmark. Following a similar idea, MA3C [124] maintains a population of agents to attack the communication channel of cooperative MARL. The individuals are improved by MATD3 and the most novel individual based on policy representation is retained. MA3C demonstrates robust communication across various tasks. EPC [125] addresses large-scale multi-agent problems by progressively expanding from small to large-scale scenarios in a curriculum-based manner. Larger-scale policies are directly cloned from the policies of the previous scale, but policies that perform well in one scale may not necessarily be suitable for the next larger scale. Thus EPC trains multiple parallel policies at new scales and uses MADDPG as a mutation operator for improvement, ultimately retaining the best-performing policies. In the particle-world environment, EPC demonstrates its ability to efficiently solve large-scale MARL problems. MAPPER [126] focuses on the multi-agent path planning problem in mixed dynamic environments. MAPPER employs EAs to enhance the stability of RL training by maintaining superior individuals and eliminating inferior ones. The mutation operator uses RL to improve the individuals. Each policy can be replaced by another policy with a certain probability that decreases as the score increases. MAPPER uses A2C instead of MARL algorithms.

A related line of work to Evo-Meta RL uses hardware acceleration to make the inner loop training far faster and more tractable [77]. By doing this, ‘Learned Policy Optimisation’ [77] and ‘Temporally-Aware Learned Policy Optimisation’ [78] discover alternative on-policy RL algorithms that significantly outperform PPO on evaluation tasks. Hardware-accelerated evolutionary meta-RL has also been applied to evolve reward functions [182], offline datasets [183], and adversarial environments [184].

Challenges and Future Directions: The utilization of EAs to assist RL through parameter search, action selection, hyperparameter tuning, and other aspects has demonstrated

the potential to further improve the performance of RL. However, utilizing EAs for RL optimization still faces several challenges: 1) Researchers require domain knowledge to define the individual form, fitness function, variation, and selection operators. 2) EAs often require a substantial number of evolutionary iterations. This takes additional computational costs and may introduce extra sample costs. 3) EAs introduce additional hyperparameters, complicating the application and tuning. 4) The works involving EA-assisted RL primarily demonstrate their efficiency in SDP. However, there is a lack of validation and systematic analysis in other problems, e.g., CTOP. In the following, we outline several prospective research directions for the future: 1) Establishing an automated configuration mechanism for EAs to enhance their usability. 2) Enhancing the efficiency of EAs, such as constructing a more sample-efficient population evaluation method. 3) Introducing EAs that are more efficient, robust, and less sensitive to hyperparameters. 4) The role of EAs in RL, beyond the aforementioned branches, remains open for further exploration. 5) There is a need for deeper investigation into the potential of EA-assisted RL in various other optimization problems.

IV. RL-ASSISTED OPTIMIZATION OF EA

In this section, we move on to the opposite side and present a comprehensive overview of RL-assisted EA. The optimization schematic is illustrated in Figure 4b. In this optimization process, the related algorithms revolve around EAs, with RL playing a supporting role in refining this process. RL cannot optimize independently. We categorize the related works into six branches based on the impact of RL on different stages of EAs. These branches include RL-assisted population initialization, RL-assisted Population Evaluation, RL-assisted Variation Operators, RL-assisted Operator Selection, RL-assisted Dynamic Hyperparameter Configuration, and Others. Each branch focuses on utilizing RL to improve specific stages of EAs, with a dedicated focus on solving distinct problems. Specifically, Population Evaluation, Variation Operators and Others primarily focus on SDP. Among these, Population Evaluation primarily focuses on leveraging the discriminative capability of RL value functions to evaluate individuals, aiming to enhance the sample efficiency of EAs. Variation Operator concentrates on using RL value functions to provide directional guidance for more efficient variation. Others primarily aim to enhance the evolutionary efficiency with RL or improve the accuracy of fitness for planning using the RL value function. Population Initialization, Dynamic Operator Selection, and Dynamic Hyperparameter Configuration primarily focus on CTOP, COP, MOOP, etc. Among these, Population Initialization primarily aims to leverage RL’s learning capability to provide initial solutions for the population. Dynamic Operator Selection focuses on the issue of operator sensitivity in EAs, i.e., no single operator can perfectly solve all problems. The goal is to use RL to achieve automated operator selection, enhancing the robustness of EAs. Dynamic hyperparameter configuration primarily focuses on utilizing RL to automatically configure the algorithm’s hyperparameters in EAs. Below, we introduce each branch of this direction along with the related algorithms.

TABLE II
RL-ASSISTED OPTIMIZATION OF EA.

Class	Algorithm	Problems	EA	RL	Detail
Population Initialization	NGGP [114]	SRP	GP	PG	Provide individuals with PG
	RL-guided GA [115]	COP	GA	PPO	Provide individuals with PG
	DeepACO [116]	COP	ACO	REINFORCE	Provide individuals with PG
Population Evaluation	SC [31]	SDP	EA	DDPG	Use Critic and Replay Buffer
	PGPS [68]	SDP	CEM	TD3	Use Critic and Replay Buffer
	ERL-Re ² [32]	SDP	EA	DDPG, TD3	H -step Bootstrap
Variation Operator	GPO [5]	SDP	GA	PPO or A2C	Apply PG for mutation
	CEM-RL [102]	SDP	CEM	TD3	Apply PG for mutation
	CEM-ACER [103]	SDP	CEM	ACER	Apply PG for mutation
	PBRL [104]	SDP	GA	DDPG	Apply PG for mutation
	NS-RL [105]	SDP	NS	DDPG	Apply PG for mutation
	DEPRL [106]	SDP	CEM	TD3	Apply PG for mutation
	QD-RL [107]	MOOP & SDP	Map-Elites-like	TD3	Guide search with QD Critics
	PGA-ME [108]	MOOP & SDP	Map-Elites	TD3	Half with PG and Half with EA
	GAC QD-RL [109]	MOOP & SDP	Map-Elites	SAC & DroQ	Half with PG and Half with EA
	CMA-MEGA [110]	MOOP & SDP	Map-Elites	TD3	Optimize the fitness with PG
	CCQD [111]	MOOP & SDP	Map-Elites	TD3	Half with PG and Half with EA & Construct the shared representations
	RefQD [112]	MOOP & SDP	Map-Elites	TD3	Half with PG and Half with EA & Construct the shared representations
	Wuji [113]	MOOP	MOEA	A2C	Apply PG to offspring
Dynamic Operator Selection	RL-GA(a) [87]	COP	GA	Q-learning	Variation operators, parent types
	RLEP [88]	CTOP	ES	Q-learning	Variation operators
	EA+RL [89]	COP	EP	Q-learning	Fitness Function
	EA+RL(O) [90]	COP	GA	Q-learning	Variation operators
	RL-GA(b) [91]	COP	GA	Q-learning	Variation operators
	GSF [92]	COP	GA	DQN, PPO	All operators during all stages
	MARLwCMA [93]	CTOP	DE CMA-ES	Q-learning	Variation operators
	MPSORL [94]	CTOP	PSO	Q-learning	Variation operators
	DEDQN [95]	CTOP	DE	DQN	Variation operators
	DE-DDQN [96]	CTOP	DE	DDQN	Variation operators
	RL-CORCO [97]	CTOP	DE	Q-learning	Variation operators
	RL-HDE [98]	CTOP	DE	Q-learning	Variation operators, parameters
	DE-RLFR [99]	MOOP	MODE	Q-learning	Variation operators
	LRMODE [100]	MOOP	MODE	Q-learning	Variation operators
	MOEA/D-DQN [101]	MOOP	MOEAs	DQN	Variation operators
Dynamic Hyperparameter Configuration	AMODE-DRL [30]	MOSP	MODE	DDQN,DDPG	Variation operators, parameter
	AGA [79]	MMOP	GA	Q-learning , SARSA	Crossover rate, mutation rate, tournament size, population size
	LTO [80]	BBOP	CMA-ES	GPS [185]	Mutation step-size parameter
	RL-DAC [81]	WBOP	-	Q-learning, DDQN	Formalize DAC as MDP
	REM [82]	CTOP	DE	VPG	Scale factor, crossover rate
	Q-LSHADE & DQ-HSES [83]	CTOP	LSHADE HSES	Q-learning , DQL	The switching time
	MADAC [84]	MOOP	MOEA/D	VDN	Hyperparameter, operators
	qlDE [85]	CTOP	DE	Q-learning	Scale factor, crossover rate
Others	RLDE [86]	CTOP	DE	Q-learning	Scale factor
	RGP [75]	SDP	GP	Q-learning	Improve efficiency
	GNP-RL [76]	SDP	GNP	Q-learning	Improve efficiency
	Grad-CEM [72]	SDP	CEM	SGD	Improve CEM efficiency
	LOOP [73]	SDP	CEM	SAC	Enhancing planning
	TD-MPC [74]	SDP	CEM	DDPG	Enhancing planning

Population Initialization. Population initialization is the initial step for all EAs, where solutions are randomly or heuristically provided as initial candidates. A well-designed population initialization can significantly enhance the search efficiency of EAs, while poor initialization can hinder the algorithms from finding superior solutions. Some works have shown that leveraging known high-quality initial solutions can greatly improve algorithm performance [186]. Consequently, several works propose to leverage RL to improve the quality of the initial population.

To address the Symbolic Regression Problem (SRP), NGGP [114] employs a policy-gradient-guided sequence generator for population initialization. Subsequently, a certain number of GP iterations are conducted, and the top E individuals from the population are combined with the initial population to train the sequence generator. Subsequently, the next iteration process begins. Experiments demonstrate that NGGP outperforms previous algorithms on the SPR benchmark. Similarly, RL-guided GA [115] employs PPO to master and match problem rules and constraints. Then the trained policies are used for population initialization. In each generation, RL learns to get new policies, which are added to the population. Experimental results demonstrate that RL efficiently learns the rules to generate candidate solutions for nuclear fuel assembly optimization problems.

Differing from the aforementioned approaches that directly employ RL to provide initial solutions, DeepACO [116] utilizes a graph neural network trained with REINFORCE to initialize the heuristic measure of Ant Colony Optimization (ACO). In conventional ACO, the heuristic measure is typically predefined based on expert knowledge. DeepACO uses the probability of RL selecting each node to initialize the heuristic measure, avoiding the introduction of expert knowledge and simultaneously accelerating the solving efficiency. DeepACO brings significant performance gains on nine combinatorial optimization problems, such as TSP, SOP, and others.

RL-assisted Population Evaluation. The population evaluation is a crucial step in obtaining fitness. However, when applying EAs to address sample-cost-sensitive problems, such as robot control, the evaluation process requires applying each solution in the population to the problem to obtain fitness. This typically incurs a substantial sample cost. To solve the problem, several works propose using RL value functions to evaluate the fitness of EA individuals, thereby improving sample efficiency.

SC [31] utilizes the expected Q values estimated using RL critics and experiences as the fitness surrogate to evaluate the population. It proposes two approaches: 1) Probabilistic evaluation using the surrogate. 2) Selecting a population twice the size of the original and filtering half using the surrogate. PGPS [68] adopts the second approach of SC for population evaluation. ERL-Re² [32] introduces H -step bootstrap for population evaluation. Each interacts with the environment for H steps, and then the value function is used to estimate the Q value of $(H + 1)$ th state, which are then summed with the extrinsic rewards in the form of cumulative discount to serve as the fitness. ERL-Re² probabilistically applies H -

step bootstrap. All the above methods have shown significant improvements in the sample efficiency. Strictly speaking, the aforementioned works should be classified as instances of synergistic optimization of EA and RL. But in this context, we focus on the population evaluation using RL. We provide a detailed explanation of these methods in section V.

RL-assisted Variation Operators. Traditional variation operators are typically gradient-free and rely on random search, which requires extensive exploration to find feasible solutions, resulting in low exploration efficiency. To improve efficiency, some works incorporate the policy gradient guidance into EAs to assist with variation operations. These works can be categorized into two main classes: Single-Objective Optimization and Quality-Diversity Optimization.

(1) *Single-Objective Optimization.* This category of algorithms aims to find solutions that maximize a single objective. GPO [5] devises gradient-based crossover and mutation by policy distillation and policy gradient algorithm. CEM-RL [102] employs the TD3 critic to guide the CEM mutation. In CEM-RL, half of the population is randomly selected and used to optimize the TD3 critic. The policy gradient from the critic is then injected into these selected individuals for mutation. The other half of the population conducts policy search by adding Gaussian noise. CEM-RL outperforms CEM and TD3 in three tasks of OpenAI MuJoCo. Another algorithm in this category is CEM-ACER [103], which follows a similar framework as CEM-RL but replaces TD3 with ACER. PBRL [104] is similar to CEM-RL but with a key difference of replacing CEM with a GA while incorporating DDPG for mutations. Specifically, PBRL allows individuals to interact with the environment for some steps and performs individual gradient optimization using a blend of experiences from the current individual and others. Furthermore, PBRL presents an automated hyperparameter tuning version called Hyperparameter tuning PBRL. In this version, each individual in the population is associated with a corresponding hyperparameter. Gaussian perturbations are added for hyperparameter mutation. The improvement using the current hyperparameter is used as the fitness. PBRL and Hyperparameter tuning PBRL demonstrate superior performance over GPO and DDPG in four MuJoCo tasks. Instead of maximizing performance, NS-RL [105] integrates DDPG to improve the exploration capabilities. In NS-RL, the fitness is defined as the L2 distance between a policy and the k -nearest policies to it within the behavior characterization space. The most novel individual is selected as the elite, while the less novel individuals are improved by minimizing the difference in their novelty compared to the elite. Furthermore, NS-RL takes the goal as the additional input of the critic to enhance the generalization. NS-RL demonstrates its efficiency on maze tasks. DEPRL [106] considers the diversity of policies, but the ultimate objective remains to maximize the rewards. DEPRL follows CEM-RL and employs Maximum Mean Discrepancy (MMD) to measure the distance among policies as the diversity metric. By concurrently maximizing rewards and MMD with gradient optimization and taking rewards and MMD as the fitness metric, DEPRL improves the diversity and exploration capabilities of the population. DEPRL outperforms CEM-RL

in some MuJoCo tasks.

(2) *Quality-Diversity Optimization*. This category of algorithms diverges from Single-Objective Optimization, considering two ultimate objectives: solution quality and solution diversity. QD-RL [107] introduces two TD3 critics into the QD framework, one for quality and the other for diversity. The calculation method of diversity is the same as NS-RL. QD-RL maintains an archive to save all past policies. At the start of each iteration, QD-RL selects individuals from the diversity-quality Pareto front constructed from the archive. Half of the selected individuals are optimized using the quality critic, and the other half are optimized using the diversity critic. Finally, the offspring are evaluated and inserted into the archive. QD-RL outperforms TD3 and other QD methods in exploration and deceptive reward maze tasks. PGA-ME [108] follows a similar process but differs in that half of the population is mutated using the original operator of Map-Elites, while the other half employs the TD3 critic for mutation. PGA-ME outperforms QD-RL and other QD methods in QDMuJoCo tasks. Furthermore, another study [187] highlights the decisive role of the policy-gradient variation operator in PGA-ME, particularly in the early optimization stages. Moreover, the study shows that PGA-ME demonstrates robust performance in both deterministic and stochastic environments, with solutions found in stochastic settings proving highly reproducible. GAC QD-RL [109] proposes a general framework by integrating SAC and DroQ [188] into PGA-ME. It reveals three insights: enhancing the update-to-data ratio (UTD), which represents the frequency of updating the critic when new transitions are collected, leads to improved performance; Unlike DroQ, the diverse data distribution in QD-RL makes it difficult for the critic to overfit, hence the regularization term in DroQ is not necessary; PGA-ME (SAC) is more efficient than PGA-ME (TD3) in tasks with low-dimensional behavioral descriptor space, while its performance is inferior in tasks with high-dimensional behavioral descriptor space. CMA-MEGA [110] maintains a distinct policy for training instead of sampling from the archive. It utilizes the features of the behavioral description space as diversity metrics. CMA-MEGA estimates the gradients of the diversity metric through OpenAI-ES, the gradients of the quality through TD3, and generates gradient coefficients for the population via CMA-ES to regulate the degree of optimization towards different objectives. Following this, it evaluates individuals within the population optimized with varying coefficients, incorporates them into the archive, prioritizes individuals based on whether new grids are filled in the archive or if grids are elevated, and then updates the CMA-ES. Lastly, optimize the TD3 critic. However, CMA-MEGA performs worse than PGA-ME on QDMuJoCo. CCQD [111] draws inspiration from ERL-Re² [32] by similarly decomposing the policy into shared representations and independent policy representations to facilitate knowledge sharing. Unlike ERL-Re², CCQD maintains multiple shared state representations to construct different knowledge spaces, enhancing the algorithm through a cooperative evolution approach. Each policy requires a unique combination of state representations and policy representations. Based on different shared representations, the behaviors of policies may vary significantly.

Policies discovered during the learning process are inserted into the archive. CCQD outperforms previous QD algorithms, reaching a new state-of-the-art performance on QDMuJoCo. RefQD [112] also employs the shared state representation and attempts to address the mismatch between old and new policies introduced by the shared state representation in Map-Elites. It periodically re-evaluates the archive and weakens the elitist mechanism of QD by maintaining more decision parts in each archive cell. RefQD outperforms PGA-ME under limited resources. Wuji [113] uses A2C to further enhance the offspring produced by crossover and mutation in MOEA, which can be seen as an additional variation operator. Wuji demonstrates superior performance in game bug detection.

RL-assisted Dynamic Algorithm Configuration. The utilization of EAs faces several significant challenges during both the configuration and application stages. Firstly, no single EA operator can efficiently solve all problems, leading to the need for a selection of EA operators based on problem characteristics and expert insights. Secondly, EAs are highly sensitive to hyperparameters, demanding meticulous adjustments. Even slight changes can lead to significant performance differences. To solve the problems, many works improve the usability and robustness of EAs by dynamically selecting the operators and tuning hyperparameter configuration, which is commonly referred to as Dynamic Algorithms Configuration (DAC). In this context, our focus lies primarily on how RL can assist in the DAC process. We categorize these works into two major domains: Dynamic Operator Selection and Dynamic Hyperparameter Configuration.

Dynamic Operator Selection. The algorithms discussed here primarily focus on COP, MOP, MOOP, and CTOP. RL-GA(a) [87] employs $Q(\lambda)$ to enhance GA by dynamic operator and parent type selections. The population itself forms the states and the rewards are defined as the improvement of the offspring compared to the parents. RL selects crossover and mutation operators along with specifying the parent types to which these operators should be applied. RL-GA(a) outperforms GA on the Traveling Salesman Problem. RLEP [88] employs Q-learning to dynamically select four mutation operators for EP. RLEP defines the rewards as the improvements of offspring over parents and directly approximate expected returns for the four mutation operators. RLEP outperforms or performs equivalently to the four basic mutation operators on functional optimization problems. EA+RL [89] employs RL to dynamically select the fitness function to enhance the optimization efficiency of GA under the target fitness. The rewards are defined as the performance differences between the best individuals under the target fitness at sequential generations. The states are constructed based on the fitness values of the population. EA+RL demonstrates improvements over ES in the Royal Roads problem and H-IFF optimization problem. EA+RL(O) [90] dynamically employs Q-learning to select crossover and mutation operators for the next generation. Similarly to EA+RL, the rewards are defined as the differences in performance between the best individual and its predecessor in the previous generation. The RL states are tailored for different tasks. The efficiency of EA+RL(O) is validated on the H-IFF optimization problem and the Traveling Salesman

Problem. RL-GA(b) [91] employs RL to enhance GA in the electromagnetic detection satellite scheduling problems. The definition of rewards is consistent with that of RL-GA(a), and the states are formulated by considering the fitness improvement and the original fitness values.

GSF [92] employs DQN and PPO to dynamically select appropriate combinations of algorithmic components (i.e., evolution operators) during different optimization stages in the capacitated vehicle routing problem with time window (CVRPTW) [189]. GSF encodes essential information required to solve the CVRPTW problem as states, including fitness, the number of vehicles, and capacity. The rewards are determined by the performance improvement of the current population compared to the initial population. GSF demonstrates the efficiency of both PPO-GSF and DQN-GSF in the CVRPTW. MARLwCMA [93] proposes a framework that combines multiple optimization algorithms, including multi-operator DE and CMA-ES. In this framework, multi-operator DE dynamically selects mutation operators with the assistance of RL. The rewards are defined as the cumulative performance improvement of offspring generated using the selected operators compared to their parents. The states contain two variables designed to reflect population diversity and quality. MARLwCMA outperforms other EAs on multiple CEC benchmarks. In MPSORL [94], the action space consists of four strategies, while states are divided unevenly into five grades based on fitness values. Subsequently, MPSORL selects the optimal strategy for each particle. To update the Q-table, a reward of 1 is returned if the particle improves; otherwise, a reward of 0 is returned. MPSORL demonstrates superior performance compared to other PSO algorithms on the CEC benchmark. DEDQN [95] utilizes DQN to dynamically select from three mutation operators in DE, primarily divided into two stages. In the first stage, DQN is trained, where states are constructed based on the information from fitness landscape theory [190], including fitness distance correlation and ruggedness of information entropy. The rewards are constructed from the live algebraic and the individual evolutionary efficiency. In the second stage, the trained DQN selects mutation operators to improve DE. DEDQN demonstrates its superiority over five well-known DE variants in the CEC2017 benchmark. DE-DDQN [96] utilizes DDQN to dynamically select mutation operators for each parent in DE. The RL state space comprises a 99-feature vector to capture the DE's current state. The reward function takes three forms: R1, reflecting the fitness differences between offspring and parents; R2, assigning a higher reward for improvements over the best solution compared to improvements over the parents; and R3, concurrently maximizing offspring's fitness differences while minimizing the gap between offspring and the optimal solution. DE-DDQN demonstrates superior performance over other DE methods and dynamic operator selection methods in the CEC benchmark. RL-CORCO [97] employs Q-learning to dynamically select two mutation operators for DE in constrained optimization problems. The population is divided into nine subpopulations based on the objective value and the degree of constraint violation, resulting in nine distinct states. Whenever a mutation strategy is applied and it either

improves or maintains the performance of the population individuals, a reward of 1 is returned; otherwise, a reward of 0 is given. Additionally, RL-CORCO incorporates a population reinitialization mechanism to prevent it from becoming trapped in local optima. RL-CORCO demonstrates its superiority over other baseline methods on the CEC benchmark.

RL-HDE [98] employs Q-learning to dynamically select six mutation operators and adjust two trigger parameters for DE. To select mutation operators, RL-HDE divides the population into 20 states based on diversity and average performance relative to the initial population. A reward 1 is given if a better solution is obtained with the selected mutation operator, 0 if there is no change, and -5 if the performance worsens. To balance global exploration and local exploitation, RL-HDE dynamically adjusts two hyperparameters and constructs six states based on a similar definition for operator selection. Regarding rewards, a better solution yields a reward of 1, no change results in a reward of 0, and deterioration leads to a reward of -1. Experimental results demonstrate that RL-HDE outperforms other baselines in solving complex interplanetary trajectory design problems such as Cassini2 and Messengerfull. DE-RLFR [99] employs Q-learning to dynamically select one of three mutation operators for each individual in MOOP. Specifically, DE-RLFR categorizes the fitness of each objective into three levels based on their ranking, resulting in nine states for RL. A reward of 10 is assigned when offspring outperform their parents; otherwise, 0 is returned. Experiments across eleven multimodal multi-objective optimization problems demonstrate that DE-RLFR can effectively construct the superior Pareto front. LRMODE [100] integrates the findings from a local landscape topology analysis with RL to approximate the optimal probability distribution for dynamically selecting MODE's mutation operators. LRMODE demonstrates superiority over other multi-objective optimization algorithms on multi-objective optimization tasks. MOEA/D-DQN [101] utilizes parent solutions and weight vectors as RL states and constructs RL rewards based on fitness improvements. It employs DQN to choose variation operators for MOEAs, leading to superior performance compared to other MOEAs across a diverse range of MOP benchmarks. AMODE-DRL [30] dynamically selects five mutation operators and adjusts two continuous parameters in multi-objective scheduling problems (MOSP). It leverages DQN to select mutation operators and DDPG to fine-tune continuous parameters. The RL states involve the current individual's fitness, fitness improvement, and population diversity. The RL rewards are defined by individual fitness and population diversity. Experimental results in both randomly generated instances and real-world problem cases demonstrate that DRL significantly enhances MODE's exploration and learning efficiency.

Dynamic Hyperparameter Configuration. In this context, we introduce the RL-assisted EA hyperparameter configuration, including crossover probability, mutation rate, population size, and others. AGA [79] leverages Q-learning to dynamically adjust the EA's crossover rate, mutation rate, tournament size, and population size. The RL states correspond to population information, e.g., maximum fitness, mean fitness, and the previous action vector. The reward function is defined as the

improvement of the best fitness. The experiments demonstrate that AGA outperforms GA on the Multimodal Problem Generator introduced by Spears [191]. LTO [80] utilizes GPS [185] for dynamically adjusting the mutation step-size parameter of CMA-ES. The RL states include the current step-size value, the current cumulative path length, the history of objective value changes, and the step-size history from the previous h iterations. LTO constructs the RL rewards based on the objective value of the current solution. LTO demonstrates its efficiency in the BBOB-2009 benchmark [192]. RL-DAC [81] formalizes DAC as a contextual MDP to enable RL to learn across a set of instances. It also introduces white-box benchmarks to demonstrate the efficiency of RL in hyperparameter tuning. Strictly speaking, RL-DAC is not limited to EAs; it can apply to all optimization algorithms. REM [82] employs Variational Policy Gradient to continuously adapt the DE's scaling parameter and crossover rate. REM uses the present population information and the corresponding randomness as the state. The reward gives 0 if the algorithm reaches the maximum generation. Alternatively, it provides the negative logarithm of the smallest function value discovered by the EA. Experiments demonstrate that DE and adaptive DE, with tuned hyperparameters, outperform counterparts and other methods.

In hybrid EAs, the timing of switching between different EA phases is crucial for algorithm performance. Different from rule-based switching methods, Q-LSHADE and DQ-HSES [83] propose an adaptive framework based on RL to adjust the switching time. Specifically, Q-LSHADE combines Q-learning and LSHADE [193], adaptively controlling when to use the linear population size reduction (LPSR) technique within L-SHADE. DQ-HSES combines DQN and HSES [194], adaptively controlling when to transition from the univariate sampling phase to the CMA-ES algorithm. The experiments in the CEC 2014 and 2018 benchmarks demonstrate that the proposed algorithms outperform SOTA EAs. MADAC [84] emphasizes the heterogeneity among various hyperparameters and recognizes that applying a single RL algorithm for configuring all parameters can introduce complexities. Hence, MADAC applies a typical MARL method value-decomposition networks (VDN) [195] to search for the appropriate settings for the multi-objective evolutionary algorithm MOEA/D's [196] four categories of hyperparameters. The RL states incorporate characteristics from different aspects, e.g., the specific problem instance, attributes associated with the ongoing optimization process, and aspects concerning the evolving population. MADAC provides rewards when the algorithm discovers better solutions than the best so far and offers greater rewards to agents that can find even better solutions in later stages. In multi-objective optimization challenges, MADAC demonstrates superior performance compared to other methods. qlDE [85] uses Q-learning to dynamically adjust two hyperparameters of DE, the scale factor F and crossover rate Cr . If the best individual in the population is better than the previous generation, the reward is 1; otherwise, it is -1. qlDE demonstrates comparable or superior performance to other DE algorithms in five truss structural weight minimization problems. RLDE [86] employs Q-learning to dynamically adjust the scale factor F of DE. Actions are

defined as 0.0, -0.1, and 0.1, which are added to the current F . If the offspring is superior to the parent, the reward is 1; otherwise, it is 0. RLDE outperforms other algorithms in the parameter extraction problems involving various PV models.

Others. Here we introduce several methods within RL-assisted Optimization of EA in other aspects. Some algorithms are influenced by the Baldwin effect and Lamarckian ideas [197], [198], which introduce learning into EAs to enhance evolutionary efficiency. The early work experimentally validates that the introduction of learning can improve the efficiency of EAs [199]. Subsequently, many works attempt to incorporate RL into EAs. RGP [75] integrates Q-learning into tree-based GP to enhance evolutionary search. RGP utilizes GP to search trees, dividing the search space into coarse-grained regions. Q-learning is embedded at the leaf nodes of the tree for decision-making. Ultimately, RGP demonstrates that incorporating Q-learning can further improve the efficiency of GP in Maze tasks. GNP-RL [76] combines GNP [200] with Q-learning. GNP leverages the higher expressive power and more compact graph structure to address the bloat issue of tree structure. GNP-RL employs RL to more fully exploit state and reward information returned by the environment, thereby enhancing optimization efficiency. Ultimately, GNP-RL demonstrates the efficiency of the method in grid environments.

In addition, some works utilize RL to enhance EA-based planning methods. Model Predictive Control (MPC) [201] is a model-based control approach that begins by designing or learning a world model. Subsequently, it employs this model to plan a sequence of actions. To enhance efficiency, several works replace traditional random sampling planning methods with CEM, such as PETS [202], PlaNet [203], and POPLIN [204]. Building upon CEM, some works incorporate RL or gradient optimization into MPC to enhance performance. In Grad-CEM [72], several random action sequences are generated, and stochastic gradients obtained from maximizing rewards based on the dynamic model are used to update the generated sequences, which improves the efficiency of CEM. Experiments in MuJoCo and DMC benchmarks demonstrate that Grad-CEM outperforms CEM. LOOP [73] combines MPC and off-policy RL. To enhance estimation accuracy, LOOP augments the traditional H-step discounted rewards with Q-values. Additionally, trajectories generated by the RL policy based on the world model are combined with those generated by CEM to optimize the CEM distribution. LOOP outperforms other model-based methods on MuJoCo tasks. TD-MPC [74] employs the same framework as LOOP, with the distinction of encoding states into a latent space for modeling the world model, learning the policy, and approximating Q-values. Experiments show that TD-MPC outperforms LOOP and SAC on DMC tasks.

Challenges and Future Directions: The above works demonstrate the efficiency of RL-assisted EAs in various aspects. Despite demonstrating the capability of RL to enhance EAs across various types of problems, RL-assisted EAs still face the following challenges: 1) Utilizing RL-assisted Optimization of EA requires researchers to have a deep understanding of the target problem to formulate it as an MDP.

Additionally, RL knowledge is necessary to select suitable algorithms for learning. 2) RL introduces extra hyperparameters, which usually need to be adjusted based on the specific problem. This may entail additional trial-and-error overhead. 3) Although RL has demonstrated the ability to enhance EAs in experiments across different branches, this lacks theoretical support and convergence guarantees. 4) Despite employing similar techniques, the absence of comparisons between different methods, especially in dynamic algorithm configuration, makes it challenging to determine which method currently outperforms in addressing specific problems. Based on the aforementioned challenges, we propose several future research directions: 1) More advanced and stable RL algorithms within the EA process require further investigation, e.g., exploring more generalized modeling approaches and developing more robust and general RL algorithms. 2) Establish theoretical guarantees for RL-assisted EAs, including convergence and performance bounds. 3) For each research branch, further investigation can be conducted to address existing limitations of current methods, e.g., develop more efficient population evaluation methods and mutation operators. 4) Researchers can construct a unified framework and evaluate the related works in a consistent benchmark, offering more valuable insights.

V. SYNERGISTIC OPTIMIZATION OF EA AND RL

The previous hybrid algorithms typically maintain only one of the approaches (EAs and RL) as the primary problem solver, while the other algorithm plays a supporting role. This section focuses on synergistic optimization algorithms that integrate the complete learning and optimization processes of RL and EAs, either (1) to simultaneously solve the same problem with collaborative mechanisms, or (2) independently optimize subproblems to obtain partial solutions, which are subsequently combined to form a complete solution. The schematics are illustrated in Figure 4c and Figure 4d. The related works in this direction focus on SDP. Below, we separately introduce two different collaboration approaches.

The first collaboration approach involves simultaneously solving the same problem using EAs and RL, with collaboration during the solving process. This collaboration approach is inspired by the complementary strengths demonstrated by EAs and RL. Specifically, EAs, based on population and random exploration, offer excellent exploration and global optimization capabilities [61]. However, random search in vast parameter space often leads to low optimization efficiency. Additionally, EAs evaluate individuals based on episodic rewards, which necessitate each individual to interact with the environment for fitness. These weaknesses result in significant sample costs, often ranging two to three orders of magnitude higher compared to RL [150]. While these coarse-grained episodic rewards make EAs more insensitive to the quality of the reward signals. In contrast, RL can leverage finer-grained information, e.g., states and rewards, and reuse historical experiences, thereby providing higher sample efficiency, yet it suffers from exploration challenges during the learning process, often prone to converging to suboptimal solutions. Moreover, RL necessitates well-designed reward signals to

ensure final performance [12]. Through the comparison above, we observe that EAs and RL each have their strengths and weaknesses. The key point of this collaboration approach is how to establish a symbiotic relationship, maintaining their respective strengths while compensating for their weaknesses. Consequently, many works try to integrate EAs with RL for synergistic optimization to enhance search efficiency and solution quality.

Single-Agent Optimization. The earliest method is ERL [61], which establishes the foundational framework. In ERL, both EAs and RL engage in policy search. EAs provide the diverse samples generated during population evaluation to RL for policy learning, thereby enhancing sample efficiency. Conversely, RL incorporates its policy into the population to participate in the evolutionary process. If the RL policy achieves better performance, it guides and facilitates population evolution. Through these mechanisms, ERL integrates the strengths of both EAs and RL. Experimental results demonstrate that ERL outperforms DDPG and GA on most OpenAI MuJoCo tasks. CERL [62] is a follow-up work to ERL, focusing on solving the sensitivity problem to RL discount factors γ . It is important to note that the role of GA in CERL is not employed for hyperparameter tuning but for policy search, which is consistent with that in ERL. Thus we discuss it in this section. CERL maintains multiple RL learners with different gammas. Unlike dynamic adjustments, CERL predefines the gamma values without tuning them in the learning process. During training, resources are dynamically allocated based on the performances of learners. Experiments on MuJoCo tasks demonstrate that CERL is more insensitive to hyperparameters. Taking inspiration from GPO [5], PDERL [63] proposes Proximal-Distilled GA (PD-GA) to address the catastrophic forgetting issue associated with GA in ERL. Specifically, PD-GA encompasses novel crossover and mutation operators. The crossover operator distills desirable behaviors from parents to offspring based on the Q values. The mutation operator adjusts the magnitude of mutations by taking into account parameter sensitivity to actions. PDERL demonstrates superior performance to ERL in OpenAI MuJoCo tasks. SC [31] focuses on mitigating the high sample cost associated with population evaluation. It proposes leveraging RL critic as a surrogate for fitness and evaluating individuals with the critic based on the samples from the replay buffer. Besides, SC introduces two mechanisms for surrogate utilization: 1) employing the surrogate for population evaluation with a probability of P , while interacting with the environment with a probability of $1 - P$; 2) generating a population larger than twice the original size and then using the surrogate model to filter half of the individuals. SC integrates with ERL and PDERL, demonstrating performance improvements on MuJoCo tasks.

Unlike previous approaches that integrate off-policy RL with EA, GEATL [64] combines on-policy RL with EA. Similar to ERL, in GEATL, RL influences EAs through policy injection. However, the influence of EAs on RL operates differently: when the elite policy of the population outperforms the RL policy, the elite policy replaces the RL policy. Moreover, if their performances are comparable, there is a 50% chance that the elite policy replaces the RL policy. GEATL demonstrates

TABLE III
SYNERGISTIC POLICY OPTIMIZATION WITH EA AND RL IN SINGLE-AGENT SETTINGS.

Both EAs and RL search in policy space							
Algorithm	Task	EA	RL	Fitness Surrogate	Policy Structure	RL Role	EA Role
ERL [61]	MuJoCo	GA	DDPG	N/A	Private	Policy Injection	Diverse Experiences For RL
CERL [62]	MuJoCo	GA	TD3	N/A	Private	Policy Injection	Diverse Experiences For RL
PDERL [63]	MuJoCo	PD-GA	DDPG	N/A	Private	Policy Injection	Diverse Experiences For RL
SC [31]	MuJoCo	GA & PD-GA	DDPG	Using Critic Estimates	Private	Policy Injection	Diverse Experiences For RL
GEATL [64]	Grid World	GA	A2C	N/A	Private	Policy Injection	Elite Policy Synchronisation
CSPS [65]	MuJoCo	CEM	PPO & SAC	N/A	Private	Policy Injection	Diverse Experiences For RL Two Separated Buffer
T-ERL [66]	MuJoCo	ES	TD3	N/A	Private	Policy Injection	Diverse Experiences For RL Two Separated Buffers
ESAC [67]	MuJoCo & DMC	A-ES	SAC	N/A	Private	Policy Crossover	Diverse Experiences For RL
PGPS [68]	MuJoCo	CEM	TD3	Using Critic Estimates	Private	Gradient Injection	Diverse Experiences For RL & Guided Policy Learning
ERL-Re ² [32]	MuJoCo & DMC	B-GA	DDPG & TD3	H-Step Bootstrap (PeVFA)	Shared	Policy Injection	Diverse Experiences For RL
VEB-RL [69]	MinAtar & Atari	GA & CEM	DQN & Rainbow	The TD Error	Private	Value Function Injection	Diverse Experiences For RL
EvoRainbow-Exp [70]	MuJoCo & Maze & MinAtar & MetaWorld	CEM	TD3 & SAC	N/A	Private	Policy Injection	Diverse Experiences For RL Genetic Soft Update
EvoRainbow [70]	MuJoCo & Maze & MinAtar & MetaWorld	CEM	TD3 & SAC	H-Step Bootstrap (Critic)	Shared	Policy Injection	Diverse Experiences For RL Genetic Soft Update
EAs search in solution space & RL search in policy space							
Algorithm	Task	EA	RL	Solution Representation		RL Role	EA Role
CORE [71]	Floorplanning	CDES	PPO	B*-Tree		Solution Injection	Corrects RL optimization

its superiority over ERL in scenarios in Grid World with sparse rewards. CSPC [65] incorporates off-policy RL, on-policy RL, and EAs. Specifically, CSPS integrates SAC, PPO, and CEM. When the SAC policy outperforms PPO or the policies in the population, it replaces those individuals. Similarly, if the PPO policy excels, it replaces the population policies. Furthermore, CSPS introduces an additional local experience buffer for SAC to store recently generated experiences and incorporates several experience filtering mechanisms. These mechanisms ensure that the added local experiences are the most recent and superior to the minimum value among all individuals at that time. SAC utilizes the local experience buffer for policy optimization with a probability of P , while utilizing the global buffer with a probability of $1 - P$. CSPS outperforms three basic algorithms on most of the MuJoCo tasks.

T-ERL [66] integrates ES with TD3 and constructs two replay buffers akin to CSPS. One buffer saves the experiences of all individuals, while the other saves recent RL experiences. T-ERL proportionally samples from both buffers for RL train-

ing. T-ERL demonstrates superiority over TD3 on three of four MuJoCo tasks ESAC [67] adopts the ERL framework while replacing DDPG with SAC and GA with a modified ES. The ES introduces an automatic adjustment mechanism to regulate the coefficient of added Gaussian noise in ES, denoted as A-ES. The coefficient is updated based on the disparity between the best performance identified within the population and the average performance. Moreover, unlike GA, ESAC does not shield elite individuals from mutation interference; instead, it employs crossover between elites and the updated ES distribution to transmit favorable traits from elites to the offspring. ESAC demonstrates its superiority over ES and other RL algorithms on MuJoCo and DMC tasks. PGPS [68] follows the ERL framework to combine CEM and TD3. It's noteworthy that PGPS maintains a full life-cycle RL policy, distinguishing itself from CEM-RL. Within PGPS, the population of size N consists of the elite from the previous generation (index 0), individuals randomly sampled from the CEM distribution (index 1 to $\frac{N}{2}$), and individu-

als selected from the large CEM-sampled pool using the surrogate mechanism from SC (index $\frac{N}{2}$ to N). Moreover, PGPS introduces Guided Policy Learning. When the behavior difference between the elites in the population and the current RL actor exceeds a threshold, behavior cloning is employed to assist RL learning. Conversely, the constraints are relaxed if the difference is within the threshold. PGPS demonstrates superior or comparable performance to CEMRL, PDERL, CERL, CEM, and some RL algorithms in MuJoCo.

ERL-Re² [32] uncovers a primary problem prevalent in the existing ERL-related research: the wide use of isolated policy architectures, where each individual operates within its private policy network. However, this independent structure often hinders the efficient transfer of valuable knowledge. To solve the problem, ERL-Re² decomposes the policies into a shared state representation and independent linear policy representations. The policy structure facilitates knowledge sharing while simultaneously compacting the policy space. Moreover, ERL-Re² proposes behavioral-level genetic operators (B-GA) based on linear policy representations, coupled with an H -step bootstrap fitness surrogate for population evaluation. ERL-Re² achieves state-of-the-art performance on MuJoCo tasks.

VEB-RL [69] addresses the issue of previous works overlooking value-based RL. VEB-RL constructs a population of value functions and corresponding target functions, using negative TD error as the fitness metric for value function evaluation. VEB-RL also introduces an elite interaction mechanism to avoid wasting interaction resources. VEB-RL significantly enhances DQN and Rainbow on MinAtar and Atari.

EvoRainbow [70] and EvoRainbow-Exp [70] systematically review this branch of works from five perspectives through experiments, providing a detailed comparison of mechanisms with the same functionality. By integrating the most effective mechanisms, they construct EvoRainbow and its exploratory version, EvoRainbow-Exp. EvoRainbow incorporates parallel mode, shared architecture, CEM, Genetic Soft Update, and H-Step Bootstrap (Critic). EvoRainbow-Exp combines parallel mode, private architecture, CEM, and Genetic Soft Update. Both EvoRainbow and EvoRainbow-Exp have demonstrated superiority over the current state-of-the-art ERL methods across 20 tasks, including locomotion tasks, manipulation tasks, Maze tasks, and Minatar.

All of the aforementioned works focus on policy search, where both RL and EAs operate in the policy parameter space to optimize policies. However, in many combinatorial optimization problems, the ultimate goal is to obtain a solution rather than a policy. In such cases, EAs are particularly suitable, as they can directly explore the solution space without relying on search in a high-dimensional policy space. To address constrained combinatorial optimization problems, CORE [71] proposes a novel EA-RL hybrid framework. In CORE, EAs conduct clustering-based evolutionary search directly in the solution space, while RL (implemented with PPO) explores the policy parameter space. To enable synergistic optimization, CORE transforms high-quality solutions found by EAs into decision sequences for RL, and uses these to construct constraint terms that steer the RL optimization direction. Conversely, RL generates solutions based on the learned policy, which

are then injected back into the EA population to enhance evolutionary progress. Experimental results show that CORE outperforms both standalone EA and RL baselines on complex floorplanning tasks.

Multi-Agent Optimization. In addition to the aforementioned single-agent optimization methods, the fusion of EAs and Multi-Agent Reinforcement Learning (MARL) has also made many advances. Here, the focus is on cooperative settings, where we need to control multiple agents to complete tasks. Compared to MARL, EAs offer additional advantages: EAs avoid the need to model the MARL problem as the MDP, thus circumventing the non-stationarity problem [60], [145]. Among these, MERL [56] is proposed to efficiently utilize both team-level and agent-level rewards for collaboration. MERL maintains a team population and optimizes the team policies using EAs with team rewards, while simultaneously optimizing individual policies using RL with agent rewards. The overall optimization process is similar to ERL. MERL demonstrates superior performance to MATD3 and MADDPG on MPE tasks. NS-MERL [57] extends MERL by considering two types of rewards during the optimization of individual rewards. To encourage exploration, NS-MERL employs a count-based method to track the number of times the current observation has been visited, where higher counts result in lower rewards. This reward is then multiplied by the original heuristic reward. Additionally, a counterfactual mechanism is utilized to calculate the contribution of each individual, thereby enhancing collaboration. The final reward is determined by multiplying the counterfactual reward with the two aforementioned rewards. Experimental results demonstrate that the constructed reward function outperforms other reward functions in the multi-rover exploration domain. CEMARL [58] shares the same idea as MERL, primarily replacing GA with CEM. In each iteration, the population teams are sampled from the CEM distribution. Subsequently, a random individual is selected and optimized using MARL based on individual rewards. Following individual optimization, all teams interact with the environment to obtain team rewards, which are then used to optimize the CEM distribution. CEMARL also maintains a policy that is soft-updated to the team with the best performance in the population, enhancing stability. Experiments show that CEMARL outperforms MERL in MPE environments. CEMARL does not maintain a full life-cycle MARL policy. Instead, it samples the MARL policy from the CEM distribution each time. Therefore, we can view MARL as the variation operator, injecting gradients into one individual of the population. As CEMARL aligns more closely with MARL settings, we include it in this branch for discussion. Different from MERL and CEMARL, EMARL [59] focuses on more general task settings, i.e., only team-level rewards. EMARL combines GA with COMA, where the population individuals are first optimized using GA, and the optimized population is further enhanced using policy gradients. EMARL is evaluated on the flocking tasks and shows better performance compared to benchmark MARL algorithms. The aforementioned algorithms are evaluated on simple tasks, whereas RACE [60] proposes a new hybrid framework and demonstrates its efficiency in facilitating col-

TABLE IV
SYNERGISTIC POLICY OPTIMIZATION WITH EA AND RL IN MULTI-AGENT SETTINGS.

Algorithm	Task and Setting	EA	MARL	EA Role	RL Role	Other features
MERL [56]	MPE (Global Information & Dense agent reward & sparse team reward)	GA	MATD3	Optimize Population with Team Reward & Provide Experiences	Optimize MARL with Agent Reward & Inject MARL Policy	Two Types of Rewards
NS-MERL [57]	Multi-rover Exploration Domain (Global Information & Dense agent reward & sparse team reward)	GA	MATD3	Optimize Population with Team Reward & Provide Experiences	Optimize MARL with Agent Reward & Inject MARL Policy	Two Types of Rewards & Exploration
CEMARL [58]	MPE (Global Information & Dense agent reward & sparse team reward)	CEM	MATD3	Optimize Population with Team Reward & Provide Experiences	Optimize MARL with Agent Reward	Two Types of Rewards
EMARL [59]	The Flocking Env (Only Team rewards Partial observation)	GA	COMA	Optimize Population with Team Reward & Provide Experiences	Optimize Population with Team Reward	Serial Optimization
RACE [60]	SMAC & MA-MuJoCo (Only Team rewards Partial observation)	A-GA	FACMAC or MATD3	Optimize Population with Team Reward & Provide Experiences	Optimize MARL with Team Reward & Inject MARL Policy	Shared Representation Architecture

TABLE V
SYNERGISTIC POLICY OPTIMIZATION WITH EA AND RL IN REWARD DESIGN.

Algorithm	Task and Setting	EA	MARL/RL	Other features
Evo-Reward [49]	Hungry–Thirsty task	PushGP	Q-learning	No LLM used
Eureka [50]	Manipulation & Locomotion	LLM-based GA	PPO	LLM-driven reward design
DrEureka [51]	Real-world Locomotion	LLM-based GA	PPO	Adds domain randomization; real-robot deployment
Roska [52]	Manipulation & Locomotion	LLM-based GA	PPO	Inter-generation policy inheritance
R* [53]	Manipulation	LLM-based GA	PPO	Reward-function crossover & parameter optimization
LaRes [54]	Manipulation & Locomotion & MinAtar	LLM-based GA	SAC & DQN	Experience sharing & continual policy learning
LERO [205]	MPE	LLM-based GA	MAPPO, VDN, and QMIX	Hybrid reward credits & observation enhancement
ReMAC [55]	Multi-Agent Manipulation	LLM-based GA	MASAC	Multi-agent reward disentanglement

laboration within complex tasks for the first time. Specifically, RACE introduces the concept of shared representations into the integration of MARL and EA. RACE divides the team policy into shared observation encoders and independent linear policy representations. RACE maximizes the value function and Value-Aware Mutual Information to inject collaboration-related information and superior global states into the shared representations. The agent-level crossover and mutation operations are operated on the linear representations to ensure stable evolution. Finally, RACE achieves superior performance compared to FACMAC, MATD3, and MERL on SMAC and MA-MuJoCo tasks.

In addition to the above collaboration approach, we can also decompose the problem into subproblems suited for EAs and subproblems suited for RL. A common pattern based on this collaboration approach is to utilize EAs for structure search and RL for policy learning. Next, we systematically introduce methods involving this collaboration approach.

Reward Design: Reward design is a crucial step in solving a task. Given a task, we typically first construct a reward

function, and then select an appropriate algorithm to solve it based on this reward function. Therefore, the overall problem-solving process can often be decoupled into two components: reward design and policy learning. The former is usually addressed using EAs, while the latter is typically handled by RL.

Early works on reward design often relied heavily on manually predefined templates, within which the reward function was searched and optimized. A representative work is Evo-Reward [49]. Evo-Reward constructs a population of reward functions and employs PushGP [206], a variant of GP, for population evolution. Experimental results indicate that Evo-Reward can discover more efficient reward functions than the original reward function on the Hungry–Thirsty task. With the rapid advancement of large language models (LLMs), recent studies have explored leveraging their coding capabilities and domain knowledge to design reward functions. Eureka [50] generates a population of reward functions using LLMs. The task is then solved using PPO, and the final policy performance is adopted as the fitness of each reward function. Based on

the reflection mechanism of LLMs, the reward population is iteratively improved. Eureka demonstrates performance that surpasses human-designed reward functions across more than ten manipulation and locomotion tasks. DrEureka [51] extends this line of work by using LLMs not only to write reward functions but also to configure domain randomization parameters, enabling effective sim-to-real transfer. ROSKA [52] incorporates both the best-performing policy and a randomly initialized policy in each iteration to maintain plasticity. Meanwhile, a Bayesian approach is used to guide policy selection. R* [53] further enhances Eureka’s performance through two mechanisms: structural evolution and parameter optimization. Structural evolution performs module-level crossover on high-quality reward functions to enable a more comprehensive search, while parameter optimization uses preference learning to fine-tune the parameters in the reward code. R* achieves consistently stronger results than Eureka. However, these prior works largely overlook sample efficiency, which is a critical metric in RL. To address this, LaRes [54] integrates off-policy RL into the reward generation process by building a shared replay buffer, where each experience is labeled with rewards evaluated by the entire reward population. In each generation, it inherits the best policy and value function, relabels the replay buffer, and applies reward scaling as well as constraint losses to avoid policy collapse. LaRes achieves superior sample efficiency and overall performance compared to Eureka, R*, and ROSKA.

Furthermore, several other works have investigated the design of reward functions under multi-agent settings. LERO [205] couples LLMs with an evolutionary outer loop to generate and refine hybrid reward functions (combining team and individual credit) and observation-enhancement functions; an inner MARL loop trains candidates and feeds back performance. On two tasks of MPE, LERO reports consistent gains and improved training efficiency. ReMAC [55] targets multi-agent manipulation with a hierarchical design: an upper LLM-driven stage maintains and iteratively optimizes a population of team-level and agent-level rewards, while a lower stage applies MARL to learn collaborative policies. Experiments on ManiCraft benchmark and show that ReMAC-generated rewards outperform human-designed rewards.

Morphological Evolution: Morphological Evolution continuously optimizes the robot morphology and the control policy. In such problems, the final solution consists of two components: the optimal morphology and its associated policy. In Morphological Evolution, EAs and RL optimize different aspects of the objective. We briefly introduce related work in this area. Classic algorithms in this category involve EAs for evolving morphologies and RL for learning policies [43]. Some works attempt to simultaneously optimize both morphology and policy using RL, such as CuCo [207] and Pre-Co [208], or simultaneously employ EAs to optimize morphology and policy, such as HyperNEAT [209], or optimize morphology with other optimization techniques, such as Bayesian optimization [43], or explore the choice of genetic encoding for morphology [48]. The hybrid algorithms in this area include EvoGym (GA) [43], HERD [44], AIEA [45], DERL [46] and TAME [47].

Interpretable AI: Policies derived from deep neural networks often lack interpretability, making them difficult to analyze and impractical for application in real-world scenarios with potential risks. To solve the problems, many works integrate decision trees with EAs and RL for highly interpretable policies. Due to some methods lacking names in the original papers, we use abbreviations based on their characteristics to denote them. POC-NLDT [37] first collects a dataset using a policy pretrained with RL and then introduces two stages: open-loop training and closed-loop training. During the open-loop training, optimization is performed using a bi-level EA [210] based on the dataset. In this process, the upper level optimizes the tree structure, while the lower level seeks the optimal values for the tree’s weights. In the closed-loop training, further re-optimization of the weights is conducted using the cumulative reward collected across several episodes. Finally, POC-NLDT demonstrates the interpretability and efficiency in four discrete action problems. GE-QL [38] evolves the tree structure using Grammatical Evolution (GE [211]) and optimizes leaf nodes using Q-learning. CG-DT [38] leverages GP to optimize structures of decision trees and employs CMA-ES [212] to optimize weights. CC-POC [39] extends POC-NLDT to continuous action spaces by constructing a population of actions. It utilizes GE for optimizing tree structures and UMDA_c^G [213] for action optimization. Q-learning is employed to optimize leaf nodes. CC-POC combines the two populations to obtain the complete solution. QD-GT [40] replaces GE with Map-Elites and defines behavioral descriptors based on action entropy and depth of decision trees. QD-GT demonstrates superior performance compared to GE schemes on the cart pole and mountain car tasks. SIRL [41] proposes a collaborative framework that constructs a population of decision trees. Actions are chosen randomly from the population to interact with the environment, and experiences are shared among them. Subsequently, each decision tree optimizes its leaf nodes using Q-learning and its tree structure using GE. SIRL demonstrates its efficiency across six MUJOCO tasks. Besides, SVI [42] is proposed to use Symbolic Regression to construct smooth analytical expression-based value functions, introducing symbolic value iteration to solve the Bellman equation. SVI offers higher interpretability compared to the black-box optimization of neural networks.

Learning Classifier Systems: Learning Classifier Systems (LCS) [214] represent a class of methods that integrate learning with evolutionary principles to discover a set of rules capable of addressing a target task. LCS can also be referred to as population-based temporal-difference methods [215]. LCS consists of four key components: (1): A population of classifiers, representing the current knowledge base. Each classifier consists of a condition, an action, and an associated fitness parameter. (2): A performance component, used to regulate the interaction between the environment and the population. (3): A reinforcement component, allocates rewards obtained to classifiers. (4): A discovery component, employed to discover new rules or refine existing ones. LCS matches related classifiers based on inputs each time. If no match is found, random classifiers are generated and added to the population. XCS [33] integrates Q-learning into LCS for learning,

where each classifier represents an action-value function, and the associated parameters correspond to the weight matrix in function approximation. Each classifier includes a condition, an action, and four main parameters. XCS utilizes fitness based on accuracy, employing GA to search in the action space for classifier selection. To improve the system’s robustness and reduce parameter dependency, researchers introduce gradient descent methods into XCS, resulting in two approaches: XCS with direct gradient (XCSG) and XCS with residual gradient (XCSRG) [215]. XCSF evolves classifiers representing piecewise linear approximations of portions of the reward surface associated with the problem solution [216]. In XCSF, the classifier’s prediction is calculated as a function, which is a linear combination of the classifier, rather than a scalar parameter. XCSF with tile coding [35] replaces the original classifier prediction function in XCSF with a tile coding approximation. DGP-XCSF [36] employs graph-based dynamical genetic programming to represent traditional condition-action production system rules for solving continuous-valued reinforcement learning problems. If you wish to explore further works related to LCS, you can refer to the work [214].

Challenges and Future Directions: Synergistic optimization has made significant progress in recent years. For instance, the early works based on the first collaboration approach are only able to achieve performance improvements on specific tasks [61]–[63]. With the development of this field, recent works can consistently outperform both EAs and RL on a wide range of tasks [32]. Despite the significant progress made in this direction, there remains a need for further investigation into how to effectively integrate the strengths of EAs and RL. The direction of synergistic optimization of EA and RL faces challenges similar to those of EA-assisted Optimization of RL and RL-assisted Optimization of EA, such as the need for domain knowledge, sensitivity to hyperparameters, and more. Concurrently, it presents a distinctive problem specific to this direction, namely, how to integrate EAs and RL to maximize the advantages of both for various problems. Currently, this direction primarily focuses on SDP. Further research is required to explore how it can complement and provide advantages for addressing other types of problems. In the future, researchers can explore in the following directions: 1) Explore how to integrate EAs and RL for synergistic optimization in addressing other problems. 2) Replace the foundational algorithms in current ERL methods with more advanced EAs or RL algorithms to fully leverage the advanced methods of both domains. 3) For the first collaboration approach, design more efficient mechanisms where EAs influence RL or RL influences EA, enhancing the positive impacts between EAs and RL. For the second collaboration approach, How to better and more automatically decompose problems to fully leverage their respective advantages is also an important research direction. In multi-agent settings, combining EAs and MARL is still at a nascent stage but holds substantial potential for advancement. Researchers can further delve into investigating how to fuse the capabilities of EAs and MARL to drive efficient collaboration.

VI. CONCLUSION

Overall, this paper systematically reviews different research directions within the field of ERL, along with the corresponding research branches within each direction. At the end of each section, the challenges faced by each direction and potential future research directions are summarized. We hope that this survey can comprehensively showcase the current development status of the ERL field to researchers, including existing algorithms, technical details, research challenges, and future research directions.

It is worth emphasizing that, although this review indicates that EA-assisted RL and the synergistic optimization of EA and RL primarily focus on SDP, RL-assisted EA optimization is geared more towards addressing other optimization problems, closely tied to the problem-solving strengths of EAs and RL. However, these directions should not be limited to specific problems. Fortunately, in recent years, there has been a considerable amount of work attempting to leverage RL to solve problems where EAs excel, such as COP [217], or employing EAs to address SDP [218], yielding significant positive outcomes. This further broadens the application boundaries of different technologies. With the development of ERL, solutions to various problems will gradually emerge in different directions, which is a development we eagerly anticipate. Therefore, our survey primarily takes a technical perspective to assist researchers in thoughtful consideration and further expansion of the application boundaries in different research directions, driving the advancement of the field.

REFERENCES

- [1] R. S. Sutton and A. G. Barto, *Reinforcement learning - an introduction*, 1998.
- [2] M. I. Jordan and T. Mitchell, “Machine learning: Trends, perspectives, and prospects,” *Science*, 2015.
- [3] M. L. Puterman, “Markov decision processes,” *HORMS*, 1990.
- [4] M. Andrychowicz, M. Denil, S. Gomez, M. W. Hoffman, D. Pfau, T. Schaul, B. Shillingford, and N. D. Freitas, “Learning to learn by gradient descent by gradient descent,” *NeurIPS*, 2016.
- [5] T. Gangwani and J. Peng, “Policy optimization by genetic distillation,” in *ICLR*, 2018.
- [6] X. Chen, C. Wang, Z. Zhou, and K. W. Ross, “Randomized ensemble double q-learning: Learning fast without a model,” in *ICLR*, 2021.
- [7] O. Vinyals, I. Babuschkin, W. M. Czarnecki, and Others, “Grandmaster level in starcraft II using multi-agent reinforcement learning,” *Nature*, 2019.
- [8] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, and S. Levine, “Residual reinforcement learning for robot control,” in *ICRA*, 2019.
- [9] L. Zou, L. Xia, Z. Ding, J. Song, W. Liu, and D. Yin, “Reinforcement learning to optimize long-term user engagement in recommender systems,” in *KDD*, 2019.
- [10] F. Ni, J. Hao, J. Lu, X. Tong, M. Yuan, J. Duan, Y. Ma, and K. He, “A multi-graph attributed reinforcement learning based optimization algorithm for large-scale hybrid flow shop scheduling problem,” in *KDD*, 2021.
- [11] J. Hao, T. Yang, H. Tang, C. Bai, J. Liu, Z. Meng, P. Liu, and Z. Wang, “Exploration in deep reinforcement learning: From single-agent to multiagent domain,” *TNNLS*, 2023.
- [12] O. Sigaud, “Combining evolution and deep reinforcement learning for policy search: a survey,” *arXiv preprint*, 2022.
- [13] T. Eimer, M. Lindauer, and R. Raileanu, “Hyperparameters in reinforcement learning and how to tune them,” *arXiv preprint*, 2023.
- [14] E. Nikishin, M. Schwarzer, P. D’Oro, P. Bacon, and A. C. Courville, “The primacy bias in deep reinforcement learning,” in *ICML*, 2022.
- [15] T. Bäck and H. Schwefel, “An overview of evolutionary algorithms for parameter optimization,” *Evol. Comput.*, 1993.

- [16] T. Bäck, *Evolutionary Algorithms in Theory and Practice: Evolution Strategies, Evolutionary Programming, Genetic Algorithms*, 1996.
- [17] P. A. Vihar, "Evolutionary algorithms: A critical review and its future prospects," in *ICGTSPICC*. IEEE, 2016.
- [18] Z. Zhou, Y. Yu, and C. Qian, *Evolutionary Learning: Advances in Theories and Algorithms*. Springer, 2019.
- [19] K. A. D. Jong, "Evolutionary computation: a unified approach," in *GECCO*, 2020.
- [20] J. Li, X. Li, and A. Wood, "Species based evolutionary algorithms for multimodal optimization: A brief review," in *CEC*. IEEE, 2010.
- [21] Y. Jin and J. Branke, "Evolutionary optimization in uncertain environments-a survey," *IEEE TEC*, 2005.
- [22] K. Han and J. Kim, "Quantum-inspired evolutionary algorithm for a class of combinatorial optimization," *TEC*, 2002.
- [23] F. P. Such, V. Madhavan, E. Conti, J. Lehman, K. O. Stanley, and J. Clune, "Deep neuroevolution: Genetic algorithms are a competitive alternative for training deep neural networks for reinforcement learning," *arXiv preprint*, 2017.
- [24] X. Yu, C. Li, and J. Zhou, "A constrained differential evolution algorithm to solve uav path planning in disaster scenarios," *KBS*, 2020.
- [25] K. Gao, Z. Cao, L. Zhang, Z. Chen, Y. Han, and Q. Pan, "A review on swarm intelligence and evolutionary algorithms for solving flexible job shop scheduling problems," *JAS*, 2019.
- [26] D. Dasgupta and Z. Michalewicz, *Evolutionary algorithms in engineering applications*, 2013.
- [27] S. Kelly and M. Heywood, "Emergent solutions to high-dimensional multitask reinforcement learning," *Evolution. Comput.*, 2018.
- [28] R. J. Smith and M. I. Heywood, "A model of external memory for navigation in partially observable visual reinforcement learning tasks," in *EuroGP*. Springer, 2019.
- [29] S. Kelly, T. Voegerl, W. Banzhaf, and C. Gondro, "Evolving hierarchical memory-prediction machines in multi-task reinforcement learning," *Genet. Program. Evolvable Mach.*, 2021.
- [30] T. Li, Y. Meng, and L. Tang, "Scheduling of continuous annealing with a multi-objective differential evolution algorithm based on deep reinforcement learning," *T-ASE*, 2023.
- [31] Y. Wang, T. Zhang, Y. Chang, B. Liang, X. Wang, and B. Yuan, "A surrogate-assisted controller for expensive evolutionary reinforcement learning," *arXiv preprint*, 2022.
- [32] J. Hao, P. Li, H. Tang, Y. Zheng, X. Fu, and Z. Meng, "Erlre\$2\$: Efficient evolutionary reinforcement learning with shared state representation and individual policy representation," in *ICLR*, 2023.
- [33] S. W. Wilson, "Classifier fitness based on accuracy," *Evol. Comput.*, 1995.
- [34] S. W. Wilson, "Classifiers that approximate functions," *Nat. Comput.*, 2002.
- [35] P. L. Lanzi and D. Loiacono, "Xcsf with tile coding in discontinuous action-value landscapes," *Evol. Intell.*, 2015.
- [36] R. J. Preen and L. Bull, "Dynamical genetic programming in xcsf," *Evol. Comput.*, 2013.
- [37] Y. Dhebar, K. Deb, S. Nagesh Rao, L. Zhu, and D. Filev, "Interpretable-ai policies using evolutionary nonlinear decision trees for discrete action systems," *arXiv preprint*, 2020.
- [38] L. L. Custode and G. Iacca, "Interpretable ai for policy-making in pandemics," in *GECCO*, 2022.
- [39] L. L. Custode and G. Iacca, "A co-evolutionary approach to interpretable reinforcement learning in environments with continuous action spaces," in *SSCI*. IEEE, 2021.
- [40] A. Ferigo, L. L. Custode, and G. Iacca, "Quality diversity evolutionary learning of decision trees," in *ACM SIGAPP*, 2023.
- [41] L. L. Custode and G. Iacca, "Social interpretable reinforcement learning," *ArXiv Preprints*, 2024.
- [42] J. Kubalík, E. Derner, J. Žegklitz, and R. Babuška, "Symbolic regression methods for reinforcement learning," *IEEE Access*, 2021.
- [43] J. Bhatia, H. Jackson, Y. Tian, J. Xu, and W. Matusik, "Evolution gym: A large-scale benchmark for evolving soft robots," *NeurIPS*, 2021.
- [44] H. Dong, J. Zhang, and C. Zhang, "Leveraging hyperbolic embeddings for coarse-to-fine robot design," *CoRR*, 2023.
- [45] S. Liu, W. Yao, H. Wang, W. Peng, and Y. Yang, "Rapidly evolving soft robots via action inheritance," *TEC*, 2023.
- [46] A. Gupta, S. Savarese, S. Ganguli, and L. Fei-Fei, "Embodied intelligence via learning and evolution," *Nat. Commun.*, 2021.
- [47] D. J. H. III, P. Abbeel, and L. Pinto, "Task-agnostic morphology evolution," *arXiv preprint*, 2021.
- [48] F. Pigozzi, F. J. C. Verdù, and E. Medvet, "How the morphology encoding influences the learning ability in body-brain co-optimization," in *GECCO*. ACM, 2023.
- [49] S. Niekum, A. G. Barto, and L. Spector, "Genetic programming for reward function search," *IEEE Trans. Auton. Ment. Dev.*, 2010.
- [50] Y. J. Ma, W. Liang, G. Wang, D. Huang, O. Bastani, D. Jayaraman, Y. Zhu, L. Fan, and A. Anandkumar, "Eureka: Human-level reward design via coding large language models," in *ICLR*, 2024.
- [51] Y. J. Ma, W. Liang, H. Wang, S. Wang, Y. Zhu, L. Fan, O. Bastani, and D. Jayaraman, "Dreureka: Language model guided sim-to-real transfer," *CoRR*, 2024.
- [52] C. Huang, Y. Chang, J. Lin, J. Liang, R. Zeng, and J. Li, "Efficient language-instructed skill acquisition via reward-policy co-evolution," in *AAAI*, 2025.
- [53] P. Li, J. Hao, H. Tang, Y. Yuan, J. Qiao, Z. Dong, and Y. Zheng, "R*: Efficient reward design via reward structure evolution and parameter alignment optimization with large language models," in *ICML*, 2025.
- [54] P. Li, H. Tang, J. Qiao, Y. Zheng, and J. Hao, "Lares: Evolutionary reinforcement learning with LLM-based adaptive reward search," in *NeurIPS*, 2025.
- [55] p. Li, T. H. Y. Yuan, and J. Hao, "ReMAC: Large language model-driven reward design for multi-agent manipulation collaboration," in *Workshop on Scaling Environments for Agents*, 2025.
- [56] S. Majumdar, S. Khadka, S. Miret, S. McAleer, and K. Tumer, "Evolutionary reinforcement learning for sample-efficient multiagent coordination," in *ICML*, 2020.
- [57] A. A. Aydeniz, R. T. Loftin, and K. Tumer, "Novelty seeking multiagent evolutionary reinforcement learning," in *GECCO*, 2023.
- [58] Y. Du, Y. Wang, Y. Cong, W. Jiang, and S. Pu, "Evolution strategies enhanced complex multiagent coordination," in *IJCNN*, 2023.
- [59] Y. Guo, X. Xie, R. Zhao, C. Zhu, J. Yin, and H. Long, "Cooperation and competition: Flocking with evolutionary multi-agent reinforcement learning," in *ICONIP*, 2022.
- [60] P. Li, J. Hao, H. Tang, Y. Zheng, and X. Fu, "Race: Improve multi-agent reinforcement learning with representation asymmetry and collaborative evolution," in *ICML*, 2023.
- [61] S. Khadka and K. Tumer, "Evolution-guided policy gradient in reinforcement learning," in *NeurIPS*, 2018.
- [62] S. Khadka, S. Majumdar, T. Nassar, Z. Dwiel, E. Tumer, S. Miret, Y. Liu, and K. Tumer, "Collaborative evolutionary reinforcement learning," in *ICML*, 2019.
- [63] C. Bodnar, B. Day, and P. Lió, "Proximal distilled evolutionary reinforcement learning," in *AAAI*, 2020.
- [64] S. Zhu, F. Belardinelli, and B. G. León, "Evolutionary reinforcement learning for sparse rewards," in *GECCO*, 2021.
- [65] H. Zheng, P. Wei, J. Jiang, G. Long, Q. Lu, and C. Zhang, "Cooperative heterogeneous deep reinforcement learning," in *NeurIPS*, 2020.
- [66] B. Zheng and R. Cheng, "Rethinking population-assisted off-policy reinforcement learning," in *GECCO*, 2023.
- [67] K. Suri, "Off-policy evolutionary reinforcement learning with maximum mutations," in *AAMAS*, 2022.
- [68] N. Kim, H. Baek, and H. Shin, "Pgps: Coupling policy gradient with population-based search," *Openreview*, 2020.
- [69] P. Li, H. Jianye, H. Tang, Y. Zheng, and F. Barez, "Value-evolutionary-based reinforcement learning," in *ICML*, 2024.
- [70] P. Li, Y. Zheng, H. Tang, X. Fu, and H. Jianye, "Evorainbow: Combining improvements in evolutionary reinforcement learning for policy search," in *ICML*, 2024.
- [71] P. Li, S. Kai, J. HAO, R. Zhong, H. Tang, Z. Tang, M. Yuan, and J. Yan, "CORE: Collaborative optimization with reinforcement learning and evolutionary algorithm for floorplanning," in *NeurIPS*, 2025.
- [72] H. Bharadhwaj, K. Xie, and F. Shkurti, "Model-predictive control via cross-entropy and gradient-based optimization," in *LADC*, 2020.
- [73] H. Sikchi, W. Zhou, and D. Held, "Learning off-policy with online planning," in *CORL*, 2021.
- [74] N. Hansen, H. Su, and X. Wang, "Temporal difference learning for model predictive control," in *ICML*, 2022.
- [75] K. L. Downing, "Reinforced genetic programming," *Genet. Program. Evolvable Mach.*, 2001.
- [76] S. Mabu, K. Hirasawa, and J. Hu, "A graph-based evolutionary algorithm: Genetic network programming (GNP) and its extension using reinforcement learning," *Evol. Comput.*, 2007.
- [77] C. Lu, J. Kuba, A. Letcher, L. Metz, C. S. de Witt, and J. Foerster, "Discovered policy optimisation," 2022.
- [78] M. T. Jackson, C. Lu, L. Kirsch, R. T. Lange, S. Whiteson, and J. N. Foerster, "Discovering temporally-aware reinforcement learning algorithms," in *ICLR*, 2024.
- [79] A. E. Eiben, M. Horvath, W. Kowalczyk, and M. C. Schut, "Reinforcement learning for online control of evolutionary algorithms," in *ESOA*, 2006.

- [80] G. Shala, A. Biedenkapp, N. H. Awad, S. Adriaensen, M. Lindauer, and F. Hutter, "Learning step-size adaptation in CMA-ES," in *PPSN*, 2020.
- [81] A. Biedenkapp, H. F. B., T. Eimer, F. Hutter, and M. Lindauer, "Dynamic algorithm configuration: Foundation of a new meta-algorithmic framework," in *ECAI*, 2020.
- [82] H. Zhang, J. Sun, Y. Wang, J. Shi, and Z. Xu, "Variational reinforcement learning for hyper-parameter tuning of adaptive evolutionary algorithm," *TETCI*, 2022.
- [83] H. Zhang, J. Sun, T. Bäck, Q. Zhang, and Z. Xu, "Controlling sequential hybrid evolutionary algorithm by q-learning," *IEEE Comput. Intell. Mag.*, 2023.
- [84] K. Xue, J. Xu, L. Yuan, M. Li, C. Qian, Z. Zhang, and Y. Yu, "Multi-agent dynamic algorithm configuration," in *NeurIPS*, 2022.
- [85] T. N. Huynh, D. T. T. Do, and J. Lee, "Q-learning-based parameter control in differential evolution for structural optimization," *Appl. Soft Comput.*, 2021.
- [86] Z. Hu, W. Gong, and S. Li, "Reinforcement learning-based differential evolution for parameters extraction of photovoltaic models," *Energy Reports*, 2021.
- [87] J. E. Pettinger and R. M. Everson, "Controlling genetic algorithms with reinforcement learning," in *GECCO*, 2002.
- [88] H. Zhang and J. Lu, "Adaptive evolutionary programming based on reinforcement learning," *Inf. Sci.*, 2008.
- [89] A. Buzdalova and M. Buzdalov, "Increasing efficiency of evolutionary algorithms by choosing between auxiliary fitness functions with reinforcement learning," in *ICMLA*. IEEE, 2012.
- [90] A. Buzdalova, V. Kononov, and M. Buzdalov, "Selecting evolutionary operators using reinforcement learning: initial explorations," in *GECCO*, 2014.
- [91] Y. Song, L. Wei, Q. Yang, J. Wu, L. Xing, and Y. Chen, "RL-GA: A reinforcement learning-based genetic algorithm for electromagnetic detection satellite scheduling problem," *Swarm Evol. Comput.*, 2023.
- [92] W. Yi, R. Qu, L. Jiao, and B. Niu, "Automated design of metaheuristics using reinforcement learning within a novel general search framework," *TEC*, 2022.
- [93] K. M. Sallam, S. M. Elsayed, R. K. Chakraborty, and M. J. Ryan, "Evolutionary framework with reinforcement learning-based mutation adaptation," *IEEE Access*, 2020.
- [94] X. Meng, H. Li, and A. Chen, "Multi-strategy self-learning particle swarm optimization algorithm based on reinforcement learning," *MBE*, 2023.
- [95] Z. Tan and K. Li, "Differential evolution with mixed mutation strategy based on deep reinforcement learning," *Appl. Soft Comput.*, 2021.
- [96] M. Sharma, A. Komninos, M. López-Ibáñez, and D. Kazakov, "Deep reinforcement learning based parameter control in differential evolution," in *GECCO*, 2019.
- [97] Z. Hu and W. Gong, "Constrained evolutionary optimization based on reinforcement learning using the objective function and constraints," *Knowl. Based Syst.*, 2022.
- [98] P. L. Y. Z., G. Dai, M. Wang, and Z. Tang, "Reinforcement learning-based hybrid differential evolution for global optimization of interplanetary trajectory design," *Swarm Evol. Comput.*, 2023.
- [99] Z. Li, L. Shi, C. Yue, Z. Shang, and B. Qu, "Differential evolution based on reinforcement learning with fitness ranking for solving multimodal multiobjective problems," *Swarm Evol. Comput.*, 2019.
- [100] Y. Huang, W. Li, F. Tian, and X. Meng, "A fitness landscape ruggedness multiobjective differential evolution algorithm with a reinforcement learning strategy," *Appl. Soft Comput.*, 2020.
- [101] Y. Tian, X. Li, H. Ma, X. Zhang, K. Tan, and Y. Jin, "Deep reinforcement learning based adaptive operator selection for evolutionary multi-objective optimization," *T-ETCI*, 2022.
- [102] A. Pourchot and O. Sigaud, "CEM-RL: combining evolutionary and gradient-based methods for policy search," in *ICLR*, 2019.
- [103] Y. Tang, "Guiding evolutionary strategies with off-policy actor-critic," in *AAMAS*, 2021.
- [104] K. W. Pretorius and N. Pillay, "Population based reinforcement learning," in *SSCI*, 2021.
- [105] L. Shi, S. Li, Q. Zheng, M. Yao, and G. Pan, "Efficient novelty search through deep reinforcement learning," *IEEE Access*, 2020.
- [106] J. Liu and L. Feng, "Diversity evolutionary policy deep reinforcement learning," *Comput. Intell. Neurosci.*, vol. 2021, 2021.
- [107] G. Cideron, T. Pierrot, N. Perrin, K. Beguir, and O. Sigaud, "QD-RL: efficient mixing of quality and diversity in reinforcement learning," *arXiv preprint*, 2020.
- [108] O. Nilsson and A. Cully, "Policy gradient assisted map-elites," in *GECCO*, 2021.
- [109] B. Lim, M. Flageat, and A. Cully, "Understanding the synergies between quality-diversity and deep reinforcement learning," in *GECCO*, 2023.
- [110] B. Tjanaka, M. C. Fontaine, J. Togelius, and S. Nikolaidis, "Approximating gradients for differentiable quality diversity in reinforcement learning," in *GECCO*, 2022.
- [111] K. Xue, R. Wang, P. Li, D. Li, H. Jianye, and C. Qian, "Sample-efficient quality-diversity by cooperative coevolution," in *ICLR*, 2023.
- [112] R. Wang, K. Xue, C. Guan, and C. Qian, "Quality-diversity with limited resources," 2024.
- [113] Y. Zheng, C. Fan, X. Xie, T. Su, L. Ma, J. Hao, Z. Meng, Y. Liu, R. Shen, and Y. Chen, "Wuji: Automatic online combat game testing using evolutionary deep reinforcement learning," in *ASE*, 2019.
- [114] T. N. Mundhenk, M. Landajuela, R. Glatt, C. P. Santiago, D. M. Faissol, and B. K. Petersen, "Symbolic regression via neural-guided genetic programming population seeding," *arXiv preprint*, 2021.
- [115] M. I. Radaideh and K. Shirvan, "Rule-based reinforcement learning methodology to inform evolutionary algorithms for constrained optimization of engineering applications," *Knowl. Based Syst.*, 2021.
- [116] H. Ye, J. Wang, Z. Cao, H. Liang, and Y. Li, "Deepaco: Neural-enhanced ant systems for combinatorial optimization," *NeurIPS*, 2023.
- [117] J. D. Co-Reyes, Y. Miao, D. Peng, E. Real, Q. V. Le, S. Levine, H. Lee, and A. Faust, "Evolving reinforcement learning algorithms," in *ICLR*, 2021.
- [118] S. Elfving, E. Uchibe, K. Doya, and H. I. Christensen, "Evolutionary development of hierarchical learning structures," *IEEE Trans. Evol. Comput.*, 2007.
- [119] Q. Liu, Y. Wang, and X. Liu, "PNS: population-guided novelty search for reinforcement learning in hard exploration environments," in *IROS*, 2021.
- [120] A. Ecoeffet, J. Huizinga, J. Lehman, K. O. Stanley, and J. Clune, "Go-explore: a new approach for hard-exploration problems," *CoRR*, 2019.
- [121] S. Chang, J. Yang, J. Choi, and N. Kwak, "Genetic-gated networks for deep reinforcement learning," in *NeurIPS*, 2018.
- [122] A. Hallawa, T. Born, A. Schmeink, G. Dartmann, A. Peine, L. Martin, G. Iacca, A. E. Eiben, and G. Ascheid, "Evo-rl: evolutionary-driven reinforcement learning," in *GECCO*, 2021.
- [123] L. Yuan, Z. Zhang, K. Xue, H. Yin, F. Chen, C. Guan, L. Li, C. Qian, and Y. Yu, "Robust multi-agent coordination via evolutionary generation of auxiliary adversarial attackers," in *AAAI*, 2023.
- [124] L. Yuan, F. Chen, Z. Zhang, and Y. Yu, "Communication-robust multi-agent learning by adaptable auxiliary multi-agent adversary generation," *arXiv preprint*, 2023.
- [125] Q. Long, Z. Zhou, A. Gupta, F. Fang, Y. Wu, and X. Wang, "Evolutionary population curriculum for scaling multi-agent reinforcement learning," in *ICLR*, 2020.
- [126] Z. Liu, B. Chen, H. Zhou, G. Koushik, M. Hebert, and D. Zhao, "MAPPER: multi-agent path planning with evolutionary reinforcement learning in mixed dynamic environments," in *IROS*, 2020.
- [127] S. Elfving, E. Uchibe, and K. Doya, "Online meta-learning by parallel algorithm competition," in *GECCO*, 2018.
- [128] M. Jaderberg, V. Dalibard, S. Osindero, W. M. Czarnecki, J. Donahue, A. Razavi, O. Vinyals, T. Green, I. Dunning, K. Simonyan, C. Fernando, and K. Kavukcuoglu, "Population based training of neural networks," *arXiv preprint*, 2017.
- [129] J. K. H. Franke, G. Köhler, A. Biedenkapp, and F. Hutter, "Sample-efficient automated deep reinforcement learning," in *ICLR*, 2021.
- [130] A. Sehgal, N. Ward, H. M. La, C. Papachristos, and S. J. Louis, "GA+DDPG+HER: genetic algorithm-based function optimizer in deep reinforcement learning for robotic manipulation tasks," in *IRC*. IEEE, 2022.
- [131] J. Grigsby, J. Y. Yoo, and Y. Qi, "Towards automatic actor-critic solutions to continuous control," *arXiv preprint*, 2021.
- [132] Y. Tang and K. Choromanski, "Online hyper-parameter tuning in off-policy learning via evolutionary strategies," *arXiv preprint*, 2020.
- [133] D. Kalashnikov, A. Irpan, P. Pastor, J. Ibarz, A. Herzog, E. Jang, D. Quillen, E. Holly, M. Kalakrishnan, V. Vanhoucke, and S. Levine, "Scalable deep reinforcement learning for vision-based robotic manipulation," in *CoRL*, 2018.
- [134] R. Simmons-Edler, B. Eisner, E. Mitchell, H. S. Seung, and D. D. Lee, "Q-learning for continuous actions with cross-entropy guided policies," *arXiv preprint*, 2019.
- [135] Y. Ma, T. Liu, B. Wei, Y. Liu, K. Xu, and W. Li, "Evolutionary action selection for gradient-based policy learning," *arXiv preprint*, 2022.
- [136] Z. Shi and S. P. N. Singh, "Soft actor-critic with cross-entropy policy optimization," *arXiv preprint*, 2021.

- [137] L. Shao, Y. You, M. Yan, S. Yuan, Q. Sun, and J. Bohg, “GRAC: self-guided and self-regularized actor-critic,” in *CoRL*, 2021.
- [138] L. Pan, L. Huang, T. Ma, and H. Xu, “Plan better amid conservatism: Offline multi-agent reinforcement learning with actor rectification,” in *ICML*, 2022.
- [139] C. S. de Witt, B. Peng, P. Kamienny, P. H. S. Torr, W. Böhmer, and S. Whiteson, “Deep multi-agent reinforcement learning for decentralized continuous cooperative control,” *arXiv preprint*, 2020.
- [140] A. Leite, M. Candadai, and E. J. Izquierdo, “Reinforcement learning beyond the bellman equation: Exploring critic objectives using evolution,” in *ALIFE*, 2020.
- [141] E. Marchesini, D. Corsi, and A. Farinelli, “Genetic soft updates for policy evolution in deep reinforcement learning,” in *ICLR*, 2021.
- [142] E. Marchesini and C. Amato, “Improving deep policy gradients with value function search,” in *ICLR*, 2023.
- [143] A. Y. Majid, S. Saaybi, T. Rietbergen, V. François-Lavet, R. V. Prasad, and C. J. M. Verhoeven, “Deep reinforcement learning versus evolution strategies: A comparative survey,” *arXiv preprint*, 2021.
- [144] Q. Zhu, X. Wu, Q. Lin, L. Ma, J. Li, Z. Ming, and J. Chen, “A survey on evolutionary reinforcement learning algorithms,” *Neurocomputing*, 2023.
- [145] M. M. Dragan, “Reinforcement learning versus evolutionary computation: A survey on hybrid algorithms,” *Swarm Evol. Comput.*, 2019.
- [146] S. Kelly and J. Schossau, “Evolutionary computation and the reinforcement learning problem,” in *Handbook of Evolutionary Machine Learning*. Springer Nature Singapore, 2024.
- [147] H. Bai, R. Cheng, and Y. Jin, “Evolutionary reinforcement learning: A survey,” *arXiv preprint*, 2023.
- [148] M. Mitchell, *An introduction to genetic algorithms*, 1998.
- [149] S. Das and P. N. Suganthan, “Differential evolution: A survey of the state-of-the-art,” *IEEE Trans. Evol. Comput.*, 2011.
- [150] T. Salimans, J. Ho, X. Chen, and I. Sutskever, “Evolution strategies as a scalable alternative to reinforcement learning,” *arXiv preprint*, 2017.
- [151] J. Lehman and K. O. Stanley, “Novelty search and the problem with objectives,” *GPTP IX*, 2011.
- [152] J. Mouret and J. Clune, “Illuminating search spaces by mapping elites,” *arXiv preprint*, 2015.
- [153] J. R. Koza *et al.*, *Genetic programming II*, 1994.
- [154] C. Watkins and P. Dayan, “Q-learning,” *Machine learning*, 1992.
- [155] R. J. Williams, “Simple statistical gradient-following algorithms for connectionist reinforcement learning,” *Machine learning*, 1992.
- [156] S. M. Kakade, “A natural policy gradient,” *NeurIPS*, 2001.
- [157] V. Konda and J. Tsitsiklis, “Actor-critic algorithms,” *NeurIPS*, 1999.
- [158] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. A. Riedmiller, “Playing atari with deep reinforcement learning,” *arXiv preprint*, 2013.
- [159] V. Mnih, K. Kavukcuoglu, D. Silver, and Others, “Human-level control through deep reinforcement learning,” *Nature*, 2015.
- [160] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, “Continuous control with deep reinforcement learning,” in *ICLR*, 2016.
- [161] S. Fujimoto, H. v. Hoof, and D. Meger, “Addressing function approximation error in actor-critic methods,” in *ICML*, 2018.
- [162] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *ICML*, 2018.
- [163] J. Schulman, S. Levine, P. Abbeel, M. I. Jordan, and P. Moritz, “Trust region policy optimization,” in *ICML*, 2015.
- [164] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint*, 2017.
- [165] E. Todorov, T. Erez, and Y. Tassa, “Mujoco: A physics engine for model-based control,” in *IROS*, 2012.
- [166] Y. Tassa, Y. Doron, A. Muldal, T. Erez, Y. Li, D. de Las Casas, D. Budden, A. Abdolmaleki, J. Merel, A. Lefrancq, T. P. Lillicrap, and M. A. Riedmiller, “Deepmind control suite,” *arXiv preprint*, 2018.
- [167] T. Rashid, M. Samvelyan, C. S. de Witt, G. Farquhar, J. N. Foerster, and S. Whiteson, “QMIX: monotonic value function factorisation for deep multi-agent reinforcement learning,” in *ICML*, 2018.
- [168] B. Peng, T. Rashid, C. S. de Witt, P. Kamienny, P. Torr, W. Böhmer, and S. Whiteson, “FACMAC: factored multi-agent centralised policy gradients,” in *NeurIPS*, 2021.
- [169] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, “Multi-agent actor-critic for mixed cooperative-competitive environments,” in *NeurIPS*, 2017.
- [170] M. A. Muñoz, Y. Sun, M. Kirley, and S. K. Halgamuge, “Algorithm selection for black-box continuous optimization problems: A survey on methods and challenges,” *Inf. Sci.*, 2015.
- [171] F. F. Peres and M. Castelli, “Combinatorial optimization problems and metaheuristics: Review, challenges, design, and development,” *Appl. Sci.*, 2021.
- [172] K. Tang, X. Yao, P. N. Suganthan, C. MacNish, Y. Chen, C. Chen, and Z. Yang, “Benchmark functions for the cec’2010 special session and competition on large-scale global optimization,” *NICAL*, 2007.
- [173] X. Li, K. Tang, M. N. Omidvar, Z. Yang, K. Qin, and H. China, “Benchmark functions for the cec 2013 special session and competition on large-scale global optimization,” *gene*, 2013.
- [174] J. Kudela, “A critical problem in benchmarking and analysis of evolutionary computation methods,” *Nat. Mach. Intell.*, 2022.
- [175] M. Flood, “The traveling-salesman problem,” *Oper. Res.*, 1956.
- [176] P. Toth and D. Vigo, *The vehicle routing problem*. SIAM, 2002.
- [177] E. Taillard, “Benchmarks for basic scheduling problems,” *EJOR*, 1993.
- [178] J. Hao, T. Yang, H. Tang, C. Bai, J. Liu, Z. Meng, P. Liu, and Z. Wang, “Exploration in deep reinforcement learning: From single-agent to multiagent domain,” *TNNLS*, 2023.
- [179] R. Rubinstein, “The cross-entropy method for combinatorial and continuous optimization,” *Methodology and computing in applied probability*, 1999.
- [180] D. Hein, A. Hentschel, V. Sterzing, M. Tokic, and S. Udfluft, “Introduction to the industrial benchmark,” *arXiv preprint*, 2016.
- [181] T. G. Dietterich, “Hierarchical reinforcement learning with the MAXQ value function decomposition,” *J. Artif. Intell. Res.*, 2000.
- [182] S. Sapora, G. Swamy, C. Lu, Y. W. Teh, and J. N. Foerster, “Evil: Evolution strategies for generalisable imitation learning,” in *ICML*, 2024.
- [183] A. Lupu, C. Lu, J. L. Liesen, R. T. Lange, and J. N. Foerster, “Behaviour distillation,” in *ICLR*, 2024.
- [184] C. Lu, T. Willi, A. Letcher, and J. N. Foerster, “Adversarial cheap talk,” in *ICML*, 2023.
- [185] S. Levine and P. Abbeel, “Learning neural network policies with guided policy search under unknown dynamics,” in *NeurIPS*, 2014.
- [186] D. Lobo and M. Levin, “Inferring regulatory networks from experimental morphological phenotypes: a computational method reverse-engineers planarian regeneration,” *PLoS Comput Biol*, 2015.
- [187] M. Flageat, F. Chalumeau, and A. Cully, “Empirical analysis of pga-map-elites for neuroevolution in uncertain domains,” *TELO*, 2023.
- [188] T. Hiraoka, T. Imagawa, T. Hashimoto, T. Onishi, and Y. Tsuruoka, “Dropout q-functions for doubly efficient reinforcement learning,” *arXiv preprint*, 2021.
- [189] O. Bräysy and M. Gendreau, “Vehicle routing problem with time windows, part II: metaheuristics,” *Transp. Sci.*, 2005.
- [190] S. Wright *et al.*, “The roles of mutation, inbreeding, crossbreeding, and selection in evolution,” *na*, 1932.
- [191] W. M. Spears, *Evolutionary Algorithms: the role of mutation and recombination*. Springer Science & Business Media, 2000.
- [192] N. Hansen, S. Finck, R. Ros, and A. Auger, “Real-parameter black-box optimization benchmarking 2009: Noiseless functions definitions,” Ph.D. dissertation, INRIA, 2009.
- [193] R. Tanabe and A. S. Fukunaga, “Improving the search performance of SHADE using linear population size reduction,” in *IEEE CEC*, 2014.
- [194] G. Zhang and Y. Shi, “Hybrid sampling evolution strategy for solving single objective bound constrained problems,” in *IEEE CEC*, 2018.
- [195] P. Sunehag, G. Lever, A. Grusl, W. M. Czarnecki, V. F. Zambaldi, M. Jaderberg, M. Lanctot, N. Sonnerat, J. Z. Leibo, K. Tuyls, and T. Graepel, “Value-decomposition networks for cooperative multi-agent learning based on team reward,” in *AAMAS*, 2018.
- [196] Q. Zhang and H. Li, “Moea/d: A multiobjective evolutionary algorithm based on decomposition,” *TEC*, 2007.
- [197] J. M. J. M. Baldwin, “A new factor in evolution,” *Diacronia*, vol. 30, 2018.
- [198] C. L. Morgan, “On modification and variation,” *Science*, 1896.
- [199] G. Hinton, S. Nowlan *et al.*, “How learning can guide evolution,” *Complex Syst.*, 1987.
- [200] K. Hirasawa, M. Okubo, H. Katagiri, J. Hu, and J. Murata, “Comparison between genetic network programming (gnp) and genetic programming (gp),” in *CEC*, 2001.
- [201] C. E. Garcia, D. M. Prett, and M. Morari, “Model predictive control: Theory and practice - A survey,” *Autom.*, 1989.
- [202] K. Chua, R. Calandra, R. McAllister, and S. Levine, “Deep reinforcement learning in a handful of trials using probabilistic dynamics models,” in *NeurIPS*, 2018.
- [203] D. Hafner, T. P. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson, “Learning latent dynamics for planning from pixels,” in *ICML*, 2019.

- [204] T. Wang and J. Ba, "Exploring model-based planning with policy networks," in *ICLR*, 2020.
- [205] Y. Wei, X. Shan, R. Miao, and J. Li, "Lero: Llm-driven evolutionary framework with hybrid rewards and enhanced observation for multi-agent reinforcement learning," in *ICIC*, 2025.
- [206] L. Spector and A. Robinson, "Genetic programming and autoconstructive evolution with the push programming language," *Genet Program Evolv Mach*, 2002.
- [207] Y. Wang, S. Wu, H. Fu, Q. Fu, T. Zhang, Y. Chang, and X. Wang, "Curriculum-based co-design of morphology and control of voxel-based soft robots," in *ICLR*, 2023.
- [208] Y. Wang, S. Wu, T. Zhang, Y. Chang, H. Fu, Q. Fu, and X. Wang, "Preco: Enhancing generalization in co-design of modular soft robots via brain-body pre-training," in *CORL*, 2023.
- [209] F. Tanaka and C. Aranha, "Co-evolving morphology and control of soft robots using a single genome," in *SSCI*. IEEE, 2022.
- [210] A. Sinha, P. Malo, and K. Deb, "A review on bilevel optimization: From classical to evolutionary approaches and applications," *IEEE TEVC*, 2017.
- [211] C. Ryan, J. J. Collins, and M. O'Neill, "Grammatical evolution: Evolving programs for an arbitrary language," in *EuroGP*. Springer, 1998.
- [212] N. Hansen and A. Ostermeier, "Adapting arbitrary normal mutation distributions in evolution strategies: The covariance matrix adaptation," in *ICEC*. IEEE.
- [213] P. Larranaga, "Optimization by learning and simulation of bayesian and gaussian networks," 1999 *EHU-KZAAIK-4/99*, University of the Basque Country, 1999.
- [214] R. J. Urbanowicz and J. H. Moore, "Learning classifier systems: a complete introduction, review, and roadmap," *J. Artif. Evol. Appl.*, vol. 2009, 2009.
- [215] M. V. Butz, D. E. Goldberg, and P. L. Lanzi, "Gradient descent methods in learning classifier systems: improving XCS performance in multistep problems," *IEEE Trans. Evol. Comput.*, 2005.
- [216] P. L. Lanzi, "Learning classifier systems: then and now," *Evol. Intell.*, 2008.
- [217] N. Mazyavkina, S. Sviridov, S. Ivanov, and E. Burnaev, "Reinforcement learning for combinatorial optimization: A survey," *Comput. Oper. Res.*, 2021.
- [218] M. K. J and T. A. W. K. H. Wray, *Algorithms for decision making*, 2022.



Pengyi Li received the B.S. degree from Tianjin University, China, in 2019. He is currently pursuing the Ph.D. degree in the College of Intelligence and Computing, Tianjin University. His current research interests are in reinforcement learning, evolutionary reinforcement learning, and multi-agent reinforcement learning.



Jianye Hao (Senior Member, IEEE) is currently an Associate Professor with Tianjin University, Tianjin, China, and the Director of the Noah's Ark Decision-making and Reasoning Laboratory, Huawei, Beijing, China. His research areas focus on reinforcement learning and multiagent systems.

The research of his team has been successfully applied in domains such as game artificial intelligence (AI), E-commerce recommendation, network optimization, and supply chain optimization. Prof. Hao has received a number of best paper awards

such as ASE2019 and CoRL2020.



Hongyao Tang received the Ph.D. degree from Tianjin University, Tianjin, China, in 2023. He currently holds a postdoctoral position at the Montreal Institute for Learning Algorithms (Mila) and the University of Montreal. His research interests include deep reinforcement learning, representation learning and continual learning.



Xian Fu received the B.S. degree from Jilin University, Changchun, China, in 2022. He is currently a master student with the College of Intelligence and Computing, Tianjin University. His current research interests are in evolutionary reinforcement learning, embodied AI, and multimodal language models.



Yan Zheng is currently an Associate Professor with Tianjin University, Tianjin, China, focuses on the research and application of intelligent decision-making theories based on the integration of reinforcement learning and large models. In recent years, his work has successfully applied large language model-driven decision agents in multiple domains, including embodied control, bioinformatics, and data science.



Ke Tang (Fellow, IEEE) received the the Ph.D. degree from the Nanyang Technological University, Singapore, in 2007. He is a Professor with the Department of Computer Science and Engineering, Southern University of Science and Technology, Shenzhen, China. His research interests mainly include evolutionary computation, machine learning, and their applications.

Dr. Tang was the recipient of the IEEE Computational Intelligence Society Outstanding Early Career Award and the Natural Science Award of Ministry of Education (MOE) of China, and was awarded the Newton Advanced Fellowship (Royal Society) and the Chang jiang Professorship (MOE of China). He is an Associate Editor for the IEEE Transactions on Evolutionary Computation and a Member of Editorial Boards for a few other journals.